



Capital University

Faculty of Computers & Artificial Intelligence

Department of Information System



E-commerce: Online Pharmacy Platform Powered by an Integrated Artificial Intelligence Assistant

A graduation project dissertation by

Ziad Mahmoud Hussein Haddad	20220187
Aya Sayed Mohamed El-Saghir	20220094
Amira Atef Mahmoud	20220078
Safi El-Din Mohamed Abdel Fattah	20220246
Shahd Ayman Amin Abdelalim	20220233
Menna Mostafa Mohamed Taha	20210966

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computers & Artificial Intelligence, at the
Information System Department, the Faculty of Computers & Artificial
Intelligence, Capital University

Supervised By:
Dr. Safaa Magdy



2026



Acknowledgment:

In successfully completing this project, many people have helped us. I would like to thank all those who are related to this project. Primarily, We would thank God for being able to complete this project with success. Then we will thank Our supervisor Dr. Safaa Magdy, under whose guidance We learned a lot about this project. His suggestions and guidance have been very helpful in various stages of project completion.



Team Members and Their Roles:



Ziad Mahmoud Hussein Haddad (Team Leader)

Ziad led the overall development of the system, coordinated between team members, and was responsible for backend development. He also contributed to frontend development and supervised the overall system integration.



Aya Sayed Mohamed El-Saghir

Aya was the originator of the project idea and contributed to its conceptualization. She was responsible for preparing the system documentation, organizing system diagrams, and also contributed to a part of the frontend development.



Amira Atef Mahmoud

Amira worked on database design and contributed to backend development. She helped in structuring the database and implementing core system functionalities.



Safi El-Din Mohamed Abdel Fattah

Safi contributed to frontend development and was responsible for developing and integrating the AI assistant into the system, including chatbot functionality and intelligent features.



Shahd Ayman Amin Abdelalim & Menna Mostafa Mohamed Taha

Shahd and Menna worked together on system analysis, testing, and quality assurance. They ensured the system meets requirements and operates efficiently.



Online Pharmacy Platform

with Integrated AI Assistant

Graduation Project Documentation

Supervised by

Dr. Safaa Magdy

Online Pharmacy Platform with AI Assistant

Graduation Project Documentation

Abstract

This project presents the design and development of a comprehensive online pharmacy platform that enables users to conveniently access medicines and healthcare products through a modern, secure digital interface. The platform allows customers to browse available medications, search and filter the catalog, manage a personal wishlist, place and track orders, and consult with registered doctors regarding their health concerns. Administrators manage products, categories, batches, expiry dates, users, and orders through a centralized dashboard, while store managers oversee inventory levels and submit stock replenishment requests, and doctors respond to patient questions and review medical histories.

The system integrates an AI-powered conversational assistant—built on top of the OpenRouter API with a multi-model fallback strategy—that delivers real-time answers to medicine-related inquiries, provides general usage guidance, and helps users discover suitable products based on natural-language descriptions of their needs. The assistant is deliberately scoped to informational guidance, while licensed doctors handle clinical questions through a dedicated consultation channel.

The platform was built around four well-defined user roles (Customer, Administrator, Store Manager, and Doctor), each with isolated permissions enforced both on the client side and on the backend. Particular attention was paid to data integrity, security, and accessibility, with features such as JWT-based authentication with automatic token refresh, role-aware route guards, batch- and expiry-tracking, allergy warnings on cart additions, namespaced per-user wishlists, dark mode with system-preference detection, and a global Ctrl+K command palette that scopes results per role.

With a focus on usability, performance, and scalability, the platform is designed to handle multiple users concurrently while maintaining a responsive and user-friendly interface across desktop and mobile form factors. The system aims to improve accessibility to pharmaceutical services and provide a reliable and efficient solution for online healthcare access.

Dedication

This project is dedicated to all individuals seeking accessible and reliable healthcare solutions through digital platforms. It is also dedicated to those who aim to improve the quality of life by leveraging technology in the pharmaceutical field.

We would like to express our sincere gratitude to **Dr. Safaa Magdy** for her continuous support, guidance, and valuable feedback throughout the development of this project. Her insights into both the academic rigor and the practical aspects of the work were invaluable, and the project would not have reached its current form without her mentorship.

We also extend our thanks to the faculty members, peers, and friends who provided constructive feedback during reviews, demonstrations, and informal discussions, and to our families for their patience and unwavering encouragement throughout the development cycle.

Table of Contents

- **Abstract** i
- **Dedication** ii
- **Table of Contents** iii

Chapter 1 — Introduction

- 1.1 Problem Statement
- 1.2 Objectives
- 1.3 Scope of Work
- 1.4 Overview of Document

Chapter 2 — Background & Literature Review

- 2.1 General Constraints
 - 2.1.1 Time
 - 2.1.2 Data Collection
 - 2.1.3 Budget and Resources
 - 2.1.4 Regulatory and Ethical Considerations
- 2.2 Project Description
- 2.3 Assumptions and Constraints
- 2.4 Background and Context
- 2.5 Project Benefits

Chapter 3 — Methodology

- 3.1 Software Tools
- 3.2 Software Requirements
 - 3.2.1 Functional Requirements
 - 3.2.2 Non-Functional Requirements
- 3.3 Software Design
 - 3.3.1 Class Diagram
 - 3.3.2 Use Case Diagram and Scenarios
 - 3.3.3 Sequence Diagram
 - 3.3.4 Activity Diagram

Chapter 4 — Implementation

- 4.1 Software Architecture
- 4.2 Database Architecture

Chapter 5 — Testing

- 5.1 Unit Testing
- 5.2 Integration Testing
- 5.3 Additional Verification

Chapter 6 — Results and Discussion

- 6.1 Results
 - 6.1.1 Expected Results
 - 6.1.2 Actual Results
- 6.2 Discussion

Chapter 7 — User Interface

Chapter 8 — Conclusion

CHAPTER 1 — INTRODUCTION

1.1 Problem Statement

In the rapidly growing digital healthcare sector, many patients and customers face difficulties in accessing pharmaceutical products in a convenient and timely manner. Traditional pharmacies require physical visits, which can be challenging for elderly individuals, patients with chronic illnesses, people living in remote or underserved areas, and customers who simply cannot reach a pharmacy during business hours. The friction of in-person visits often leads to delayed treatment, incomplete medication courses, or reliance on inadequate alternatives.

Pharmacies, on the other hand, often struggle with efficient inventory management, order tracking, and customer communication when relying on manual or partially digital systems. Spreadsheets and paper logs are error-prone, batches with overlapping expiry dates are difficult to monitor, and reorder decisions depend on the manager's memory rather than reliable data. The result is a combination of stock-outs of essential medicines and waste from drugs that expire on the shelf.

Customers frequently encounter challenges such as:

1. Difficulty in finding required medicines quickly when stock levels and pricing are not transparent online.
2. Limited access to reliable pharmacy services outside working hours, weekends, and public holidays.
3. Lack of clear information about medicine availability, dosage forms, manufacturer, active ingredients, and pricing before they travel to the store.
4. No structured channel to ask a qualified professional a quick health-related question without scheduling a clinic visit.
5. No personal record of past purchases, allergies, or chronic conditions that can be used to surface safety warnings at the moment of purchase.

At the same time, pharmacies face challenges including:

6. Inefficient management of medicine inventory, batches, and customer orders, leading to wasted product and frustrated customers.
7. Limited digital presence and customer reach, making it difficult to compete with chain pharmacies that already operate online.
8. Difficulty in maintaining organized records of customers, transactions, prescriptions, and consultations across staff and shifts.
9. Lack of automated alerts when stock falls below safe thresholds, making replenishment reactive rather than proactive.
10. No integrated channel for in-house doctors or licensed pharmacists to support customers at the moment of purchase.

This project aims to address these issues by developing a comprehensive and scalable online pharmacy web application that connects customers with pharmacies through a secure and user-friendly platform. The system is designed to improve accessibility, efficiency, and reliability for both customers and pharmacy administrators, while introducing safety features—batch and expiry tracking, allergy checks, doctor consultations, and an AI assistant—that elevate it beyond a generic e-commerce store.

1.2 Objectives

The primary objective of this project is to design and develop a comprehensive web-based Online Pharmacy system that enables users to access pharmaceutical products easily and efficiently through the internet. The system aims to provide a reliable digital solution that improves the interaction between customers and pharmacies while reducing the dependency on traditional, in-person pharmacy visits.

This project also seeks to enhance the overall pharmacy service experience by streamlining operations such as medicine management, order processing, inventory replenishment, and user account handling. By leveraging modern web technologies, the system ensures better organization, faster service delivery, and improved accuracy in handling pharmaceutical data.

The specific objectives of this project include:

- **Customer-facing storefront:** Design a simple and intuitive user interface that allows customers to browse, search, filter, and purchase medicines with minimal effort, supported by a wishlist, persistent cart, and order tracking.
- **Centralized administration:** Develop a centralized system that enables pharmacies to manage medicine inventory, categories, batches, expiry dates, and customer orders efficiently from a single dashboard.
- **Reduced friction:** Reduce delays and inconvenience associated with traditional pharmacy services by enabling customers to complete the entire purchase journey online, at any time of day.
- **Secure access:** Ensure secure handling of user data through proper authentication and access control mechanisms—specifically, JWT-based authentication with refresh tokens and role-based route guards.
- **Role separation:** Provide four well-defined user roles (Customer, Administrator, Store Manager, Doctor) with strictly enforced permission boundaries on both client and server.
- **AI assistance:** Integrate a conversational AI assistant capable of answering medicine-related inquiries in natural language, helping customers identify suitable products and learn basic usage information.
- **Doctor consultation channel:** Provide a structured Q&A workflow that allows customers to submit medical questions to registered doctors and receive professional responses before purchasing medicines.
- **Inventory safety:** Implement real-time low-stock alerts and a stock-request workflow between store managers and administrators to keep essential medicines available without overstocking.

- **Patient safety:** Implement allergy and condition checks against the customer's stored medical history at the moment a product is added to the cart, surfacing warnings before purchase rather than after.
- **Scalability and maintainability:** Provide a system architecture that can accommodate future growth and additional functionalities (online payments, prescription uploads, multi-pharmacy support, mobile applications) without requiring a structural rewrite.
- **Quality of experience:** Deliver a polished, premium feel through dark-mode support, smooth animations, accessible color contrast, responsive layouts, and a global Ctrl+K command palette that lets power users navigate the application without leaving the keyboard.

1.3 Scope of Work

This project covers the design and development of a complete web-based Online Pharmacy system that supports the essential processes involved in online pharmaceutical services. The system provides a digital platform that enables organized interaction between customers and pharmacies while maintaining clarity in system operations and responsibilities.

In scope:

- Analysis of system requirements and modeling of business processes using UML diagrams (use case, sequence, activity, and class diagrams).
- Design of the overall system structure following a three-tier architectural pattern (presentation, business logic, data).
- Implementation of both frontend and backend components: a React-based single-page application on the client side and a Django REST Framework API on the server side.
- Implementation of four distinct user roles—Customer, Administrator, Store Manager, and Doctor—each with dedicated dashboards, navigation, and permission boundaries.
- Catalog management: categories, products, batches, expiry tracking, low-stock thresholds, and stock-request workflow.
- Customer journey: browsing, search, filtering, sort, wishlist, cart, checkout, and order history.
- Doctor-customer consultation: submitting medical questions and receiving professional answers.
- Medical history management: customers maintain records of allergies, chronic conditions, and current medications.
- AI assistant: integration with the OpenRouter API for natural-language Q&A about medicines and general guidance.
- Dark mode and accessibility polish: theme persistence, system-preference detection, high-contrast color tokens, smooth animations, focus-visible styles, and keyboard-driven navigation including a global Ctrl+K command palette.
- Security: JWT authentication with refresh tokens, request/response interceptors, role-based permission classes on the backend, and password hashing.
- Documentation, testing, and deployment within a defined project timeline.

Out of scope (deferred for future iterations):

- Native mobile applications (the platform is responsive and works on mobile browsers, but no dedicated iOS/Android app is delivered).
- Integration with external healthcare information systems (HIS, EMR, hospital APIs).
- Online payment gateway integration (cash on delivery is the only payment method delivered in this iteration).
- Prescription image upload and verification by a licensed pharmacist.
- Real-time delivery tracking with map and ETA.
- Multi-pharmacy / multi-tenant support (single-pharmacy deployment for this release).
- Multilingual user interface (English-only for the initial release).

This boundary keeps the project deliverable within the available timeline while leaving the architecture open to extension.

1.4 Overview of the Document

The remainder of this document is organized as follows:

- **Chapter 2 — Background & Literature Review** introduces the project's context, the constraints under which it was developed, the assumptions made about the operating environment, and the benefits the system delivers to its different stakeholders.
- **Chapter 3 — Methodology** describes the tools used during development, the functional and non-functional requirements that drove the design, and the design artefacts (class, use case, sequence, and activity diagrams) that guided implementation.
- **Chapter 4 — Implementation** presents the technical realization of the system: the layered software architecture, the breakdown of the React frontend and the Django backend, and the database schema with the relationships between the principal entities.
- **Chapter 5 — Testing** describes the automated test suite — unit tests covering individual models, serializers, and components, and integration tests covering full HTTP request/response cycles through the API — together with the additional manual verification activities performed before each milestone.
- **Chapter 6 — Results and Discussion** compares the originally expected outcomes against the actual results obtained after implementation, deployment, and testing, and discusses what worked well, what required iteration, and what lessons were learned.
- **Chapter 7 — User Interface** documents every major screen of the system with screenshots and a description of each screen's purpose, content, layout, and the user flow it supports.
- **Chapter 8 — Conclusion** summarizes the contributions of the project, restates how the original objectives were met, and outlines a roadmap of enhancements that would be valuable in future iterations.

Specifically, the reader will find the following content in the chapters above:

- **Module Descriptions:** Detailed explanation of each major system module (user management, medicine management, order processing, batch and expiry tracking, doctor consultation, AI chat, and pharmacy administration), including the role of each module and the flow of data between them.
- **Component Interactions:** Overview of how different system components interact—how user accounts relate to medical history, how cart and order modules cooperate during checkout, how the stock-request workflow connects store managers with administrators, and how the AI assistant integrates with the rest of the application.
- **User Interface Layout:** Descriptions of the main pages and screens of the system, including their content, layout structure, and user navigation flow.
- **Functional Requirements:** A detailed list of all functional requirements expected from the system, describing the actions that users and administrators can perform within the online pharmacy platform.
- **Non-Functional Requirements:** Definitions of system quality attributes such as performance, scalability, security, usability, accessibility, and availability.
- **Use Case Diagrams and Scenarios:** Visual and textual representations of user interactions with the system, illustrating how each role uses the system to achieve specific goals.
- **Special Requirements:** Any additional technical or operational constraints related to the system, such as data privacy, security regulations, and system compatibility considerations.

CHAPTER 2 — BACKGROUND & LITERATURE

REVIEW

2.1 General Constraints

Hardware Constraints

- Availability of a computer, tablet, or smartphone capable of running a modern web browser to access the online pharmacy system.
- For the server side: a host capable of running a Python 3.10+ runtime and serving HTTP requests; the development build runs comfortably on a single workstation, while production would benefit from a dedicated server or cloud instance.

Software Constraints

- A stable Wi-Fi connection or reliable mobile data on the client side.
- A secure HTTPS-capable internet connection to ensure safe data transmission.
- Modern browsers with support for ES2020+ JavaScript (the application targets Chrome, Edge, Firefox, and Safari in their last two major versions).
- A relational database engine—SQLite is used during development for portability; the schema is migration-driven and is portable to PostgreSQL or MySQL for production.

2.1.1 Time

- The system itself imposes no time-of-day constraints, as users can access the online pharmacy at any time. Server availability is the only timing factor, and it is targeted at 99.9 percent uptime.
- The development effort was constrained to the project semester, which shaped the prioritization of features and the scope decisions documented in section 1.3.

2.1.2 Gathering Data

- A sufficient amount of data is required to operate the system efficiently.
- Medicine data, user data, category data, batch and expiry data, and order data are required for full system functionality. During development, representative seed data was created to demonstrate every major workflow.
- Where the seed catalog includes images, royalty-free assets were used and stored locally under the public assets directory; in production, the same paths can resolve to a CDN or object-storage URL without changing application code.

2.1.3 Budget and Resources

- The project was developed using only open-source components and free-tier external services, in line with academic constraints. The AI assistant uses OpenRouter's free or low-cost models with a fallback strategy across multiple providers, so the operating cost remains minimal during demonstration.

2.1.4 Regulatory and Ethical Considerations

- The platform deliberately limits the AI assistant's role to general informational responses and product guidance; clinical or prescription-related advice is routed through licensed doctors via the consultation channel. This separation reflects ethical and regulatory expectations around online health advice.
- User medical data (allergies, conditions, medications) is treated as sensitive: it is bound to the authenticated user, never shared between accounts, and exposed only on the authenticated request paths that need it.

2.2 Project Description

This project is a web-based online pharmacy system designed to provide a digital platform that connects customers with pharmacies. The system aims to simplify the process of searching for medicines and placing orders by offering a secure, reliable, and user-friendly environment.

Pharmacies can manage their accounts, add and update medicine information, monitor inventory levels, track batches and expiry dates, and handle customer orders through a centralized system. The platform enables pharmacies to maintain accurate records, anticipate replenishment needs, and ensure medicine availability without holding excessive stock that could expire unsold.

Customers, on the other hand, can browse available medicines, view detailed product information (including manufacturer, active ingredient, and prescription requirements), add items to a wishlist for later, place items in a cart, complete checkout, and track the status of their orders from approval through to delivery. Returning customers benefit from saved profiles, persistent carts, and order history.

A doctor consultation module allows customers to submit medical questions and receive answers from registered doctors, while the medical-history feature lets customers record their own allergies, chronic conditions, and current medications—data that the platform uses to surface safety warnings when an at-risk product is added to the cart.

Additionally, an administrative interface is provided to monitor system activity, manage products and categories, control user accounts, supervise orders, and approve or reject stock-replenishment requests submitted by store managers. The system is developed with scalability, security, and usability in mind and follows modern web development standards including responsive design, accessibility-aware color contrast, dark-mode support, and a global keyboard-driven command palette.

2.3 Assumptions and Constraints

Assumptions

- Users have access to stable internet connectivity and modern web browsers.
- Users possess basic digital skills to navigate the system and perform common tasks such as registration, login, and ordering.
- Users provide accurate and valid information during registration and transactions.
- Mobile users access the system through responsive web interfaces rather than dedicated native applications.
- The pharmacy operates within a single legal jurisdiction; the system does not handle cross-border regulatory differences.
- Customers reading AI-assistant responses understand that they are general informational guidance and not a substitute for medical advice.

Constraints

- The system operates strictly within a web-based environment.
- Development must be completed within the defined project timeline.
- Sensitive user data (e.g. passwords) must be securely encrypted/hashed and never stored in plain text.
- System functionality depends on server availability and internet connectivity.
- Language support is limited to English in the initial release; the architecture allows for a future internationalization layer.
- Payment is limited to "cash on delivery" in the initial release; online payment integration is left for future iterations.

2.4 Background and Context

With the increasing shift toward digital healthcare services, many individuals face challenges in accessing medicines conveniently and efficiently. Traditional pharmacy services often require physical visits, which may not be suitable for all users—especially the elderly, patients with chronic conditions, residents of remote areas, or working customers who cannot visit during business hours.

This project was initiated to address these challenges by developing a web-based online pharmacy system that provides essential pharmaceutical services through a centralized platform. The system supports medicine management, batch and expiry tracking, order processing, doctor consultation, AI-assisted guidance, and user interaction in a unified digital environment.

The system distinguishes itself from a generic e-commerce solution in several important ways. First, the catalog is organized around pharmaceutical primitives—active ingredients, manufacturers, prescription

requirements, batch numbers, and expiry dates—rather than only the marketing attributes typical of retail. Second, the inventory model captures multiple batches per product so that first-expiry-first-out (FEFO) decisions and expiry alerts are possible, instead of treating stock as a single quantity field. Third, customer profiles include a medical history that the system actively consults during checkout to warn the customer about likely allergic interactions. Fourth, the system embeds a structured doctor consultation channel so that customers can ask professional questions before they buy, rather than purchasing first and asking later. Finally, the AI assistant fills the gap for general informational questions ("what is paracetamol used for?", "what does this category contain?") that don't require a doctor but are tedious to answer through static FAQ pages.

The online pharmacy system aims to improve accessibility, enhance user experience, and support the digital transformation of healthcare services—while remaining within the boundaries of responsible, ethical online-pharmacy practice.

2.5 Project Benefits

The proposed system offers benefits across each stakeholder group.

For Pharmacies

11. Centralized management of medicines, categories, and inventory through a single web dashboard, eliminating the inconsistency of paper-based or spreadsheet-based tracking.
12. Efficient handling of customer orders with status transitions (pending → approved → processing → shipped → delivered) that mirror real fulfilment.
13. Improved organization and record-keeping, including historical orders, transaction logs, and audit trails of product changes.
14. Visibility into batches and expiry dates so stock can be rotated and shrink reduced.
15. A structured stock-request workflow between store managers and administrators that replaces ad-hoc verbal or messaging-app requests.

For Customers

16. Easy access to medicines through online browsing with category filters, full-text search, and sortable results.
17. Convenient ordering and order tracking—including a persistent cart, wishlist, and full order history bound to the customer's account.
18. Reduced need for physical pharmacy visits, especially valuable for elderly patients or those with mobility limitations.
19. Safety-aware checkout: allergies and active conditions stored in the medical history are checked when products are added to the cart, with clear warnings.

20. A direct line to registered doctors for non-urgent medical questions, and a 24/7 AI assistant for general guidance.

For System Administrators

21. Clear visibility of system usage and activity across all roles.
22. Tools to monitor system performance, data integrity, and user behavior.
23. Granular control over user accounts, roles, products, categories, batches, and orders.
24. Approval workflows for stock requests and order status transitions, supporting compliance and accountability.

For Doctors

25. A consolidated inbox of customer questions, sortable by date and answered/unanswered status.
26. Access to the patient's recorded medical history (allergies, conditions, medications) for informed responses.
27. A clean professional dashboard distinct from the customer-facing storefront.

General Benefits

- Improved accessibility to pharmaceutical services for users who cannot easily reach a physical pharmacy.
- Enhanced efficiency and reliability of pharmacy operations, with reduced manual errors and reduced expired-product waste.
- Support for the broader digital transformation of the healthcare sector, providing a template that can be extended to multi-pharmacy networks, prescription handling, and integration with delivery services.

CHAPTER 3 — METHODOLOGY

3.1 Software Tools

The project was implemented using a modern, well-supported open-source stack chosen for productivity, community ecosystem, and long-term maintainability.

Frontend Stack

- **React 19** — JavaScript library used to build a component-based, single-page user interface. React's virtual DOM and hook-based state model make it well-suited for the interactive, role-aware screens of this system.
- **React Router 6** — Declarative routing for the SPA, including private routes that gate sub-trees of the application by authentication and role.
- **Tailwind CSS 3** — Utility-first CSS framework used to maintain a consistent visual system and to support dark mode through the ``dark:`` variant. The configuration extends the default palette with the project's brand colors (teal, slate, accent hues).
- **Framer Motion 12** — Animation library used for entrance transitions, hover effects, and the smooth open/close animation of the global command palette.
- **Headless UI** — Provides accessible, unstyled primitives (combobox, dialog, transition) that are styled with Tailwind to match the application's design language.
- **react-icons** — Icon set used throughout the interface; the ``Fi`` (Feather) family is preferred for its consistent stroke and modern feel.
- **Axios** — HTTP client used for all communication with the backend. A request/response interceptor pair handles JWT attachment and automatic refresh on ``401`` responses.

Backend Stack

- **Python 3.10+** — Runtime for the backend application.
- **Django 4.2+** — The web framework used to structure the backend, providing the ORM, the admin scaffolding, the authentication system, and the URL/views layer.
- **Django REST Framework (DRF)** — Layered on top of Django to expose the data model as a JSON API. Generic viewsets and routers map cleanly to the resource model.
- **djangorestframework-simplejwt** — Provides JSON Web Token authentication with access/refresh tokens, used in combination with DRF's permission classes for role-based access control.
- **django-cors-headers** — Allows the React development server (port 3000) to call the Django development server (port 8000) during development.
- **Pillow** — Image-handling library used for product image uploads and thumbnails.

Database

- **SQLite** — Embedded relational database used during development for portability. Because Django's ORM abstracts the SQL dialect, the same schema and queries run unchanged on PostgreSQL or MySQL when promoting to production.

Third-Party Integrations

- **OpenRouter API** — Aggregator endpoint used for the AI chatbot. The integration is implemented with a multi-model fallback list so that if one upstream model is unavailable or rate-limited, the system automatically tries the next one without surfacing the failure to the user.

Development Tools

- **Git** for version control with feature-branch workflow.
- **npm** for managing JavaScript dependencies; **pip** with a `requirements.txt` lockfile for Python dependencies.
- **VS Code** as the primary editor.
- **Postman** and the browser DevTools network panel for API debugging.
- A test suite executed via React Testing Library and Jest on the frontend, and Django's built-in test runner on the backend.

3.2 Software Requirements

3.2.1 Functional Requirements

The functional requirements are organized by user role. Each requirement is presented as a function with a short description, the preconditions that must hold for the function to be invoked, and the priority assigned during planning.

1) Administrator

Field	Value
Function Name	Admin Login
Description	Administrator logs into the platform using valid credentials to access management features.
Pre-Conditions	Admin must have a valid account with the `admin` role assigned.
Priority	High

Field	Value
Function Name	Admin Logout

Description	Administrator can log out from their account safely; the JWT refresh token is invalidated server-side.
Pre-Conditions	Admin must be logged in.
Priority	Medium

Field	Value
Function Name	Manage Categories
Description	Admin can add, edit, or delete medicine categories to organize the catalog efficiently and support category-filtered browsing on the storefront.
Pre-Conditions	Admin must be logged in.
Priority	High

Field	Value
Function Name	Manage Products
Description	Admin can add new products, update existing products, or delete products. Product details include name, description, price, manufacturer, active ingredient, prescription requirement, low-stock threshold, image, and category.
Pre-Conditions	Admin must be logged in.
Priority	High

Field	Value
Function Name	Manage Batches
Description	Admin can register batches against existing products with batch number, quantity, expiry date, and cost price. Expired and soon-to-expire batches are surfaced visually in the dashboard.
Pre-Conditions	Admin must be logged in; the parent product must exist.
Priority	High

Field	Value
Function Name	Manage Users
Description	Admin can view, edit, or delete user accounts within the system. This helps control system access and maintain accurate user information. Role assignment (customer / store_manager / doctor / admin) is also part of this function.
Pre-Conditions	Admin must be logged in.
Priority	High

Field	Value
Function Name	Manage Orders
Description	Admin can view all customer orders and update the order status by approving, rejecting, processing, shipping, or marking as delivered. The function also exposes line-level item details and the customer's shipping address.
Pre-Conditions	Admin must be logged in.
Priority	High

Field	Value
Function Name	Approve Stock Requests
Description	Admin reviews stock-replenishment requests submitted by store managers, approves or rejects them, and the approval transitions the related batch into stock.
Pre-Conditions	Admin must be logged in; at least one pending stock request must exist.
Priority	Medium

Field	Value
Function Name	Edit Profile
Description	Admin can update personal profile information such as name, email, phone, address, and password. Helps keep account information up to date and secure.
Pre-Conditions	Admin must be logged in.
Priority	Medium

2) Customer

Field	Value
Function Name	Customer Register
Description	Allows a new customer to create an account by providing personal details (username, email, phone, password). The system stores the information securely (password is hashed) for future login.
Pre-Conditions	Customer must not have an existing account with the same username, email, or phone number.
Priority	High

Field	Value
Function Name	Customer Login
Description	Allows the customer to log in using email or username together with their password. Issues an access token (short-lived) and a refresh token (longer-lived) on success.
Pre-Conditions	Customer must have a registered account and enter valid credentials.
Priority	High

Field	Value
Function Name	Browse Products
Description	Allows customers to view the full medicine catalog, paginated and filterable by category, with grid/list view toggle.
Pre-Conditions	Products must be available in the system.
Priority	High

Field	Value
Function Name	Search Medicines
Description	Allows customers to search for medicines by name, manufacturer, active ingredient, or category, with case-insensitive partial matching.
Pre-Conditions	Medicines must be available in the system.
Priority	High

Field	Value
Function Name	Sort and Filter Products
Description	Customers can sort the catalog (Featured, Name A→Z, Name Z→A, Price Low→High, Price High→Low) and filter by category. Sorting is performed client-side over a single fetched page for responsiveness.
Pre-Conditions	At least one product must be available.
Priority	Medium

Field	Value
Function Name	Quick Preview Product
Description	Customers can open a quick-preview modal that shows full product details (image, description, manufacturer, ingredient, prescription badge, price, stock) without leaving the catalog.
Pre-Conditions	The product must exist.
Priority	Low

Field	Value
Function Name	Add To Cart
Description	Allows customers to add selected medicines to the shopping cart. Before adding, the system checks the customer's medical history for known allergies against the product's active ingredient and surfaces a warning if a match is found.
Pre-Conditions	Customer must be logged in.
Priority	High

Field	Value
Function Name	Manage Wishlist
Description	Customers can save items to a personal wishlist, toggle items on or off via a heart button, and review the wishlist later. The wishlist is namespaced per user and persists in browser storage with cross-tab synchronization.
Pre-Conditions	Customer is logged in (logged-out browsing uses a separate guest bucket so wishlists never leak between users on a shared device).
Priority	Medium

Field	Value
Function Name	Make an Order
Description	Allows customers to place an order for the medicines currently in their cart, supplying a shipping address and selecting a payment method (cash on delivery in this release).
Pre-Conditions	Customer must be logged in and the cart must contain at least one medicine that is in stock.
Priority	High

Field	Value
Function Name	Make Payment
Description	Allows customers to complete the payment process for their orders using the available payment method. The current release ships with cash-on-delivery; the field is captured at checkout for future online-payment integration.
Pre-Conditions	Customer must be logged in and an order must be created.
Priority	High

Field	Value
Function Name	Track Order
Description	Customers can view all their past and current orders, with status badges (Pending, Approved, Processing, Shipped, Delivered, Cancelled, Rejected) and itemized line details.
Pre-Conditions	Customer must be logged in.
Priority	High

Field	Value
Function Name	Cancel Pending Order
Description	Allows customers to cancel an order while it is still in the Pending state; the cancellation restores stock and updates the order status.
Pre-Conditions	Customer must be logged in; the order's current status must be Pending.
Priority	Medium

Field	Value
Function Name	Manage Medical History
Description	Customers can record their allergies, chronic conditions, and current medications. This information powers the allergy check during cart-add and is visible to doctors when answering the customer's questions.
Pre-Conditions	Customer must be logged in.
Priority	Medium

Field	Value
Function Name	Interact With Chatbot
Description	Allows customers to interact with the AI assistant to ask questions, get general medicine information, and receive assistance navigating the system. The assistant is backed by the OpenRouter API with multi-model fallback.
Pre-Conditions	Customer must be logged in.
Priority	Medium

Field	Value
Function Name	Ask Doctor
Description	Allows customers to send medical questions to a registered doctor for advice before purchasing medicines, and to view the doctor's reply when it arrives.

Pre-Conditions	Customer must be logged in.
Priority	Medium

Field	Value
Function Name	Edit Profile / Logout
Description	Customer can update profile information (first/last name, email, phone, address, password) and log out securely.
Pre-Conditions	Customer must be logged in.
Priority	Medium

3) Store Manager

Field	Value
Function Name	Store Manager Login
Description	Allows the store manager to sign in using email or username and password. Ensures secure access to inventory features.
Pre-Conditions	Store manager must have a valid account with the `store_manager` role.
Priority	High

Field	Value
Function Name	Monitor Stock Levels
Description	Store manager views the full product list with current stock totals; products below their `low_stock_threshold` are highlighted with a "Low Stock" badge to draw attention.
Pre-Conditions	Store manager must be logged in.
Priority	High

Field	Value
Function Name	View Batches
Description	For each product, the store manager can drill into the list of associated batches with their batch numbers, quantities, and expiry dates, including expired and soon-to-expire indicators.
Pre-Conditions	Store manager must be logged in.
Priority	Medium

Field	Value
Function Name	Submit Stock Request

Description	Allows the store manager to place a replenishment request for a low-stock product, specifying the requested quantity, reason, and either an existing batch (to top up) or a new batch number with expiry date.
Pre-Conditions	Store manager must be logged in; the product must exist.
Priority	High

Field	Value
Function Name	Track My Stock Requests
Description	Store manager views the history of their submitted stock requests, sees their approval/rejection status, and can delete obsolete requests.
Pre-Conditions	Store manager must be logged in.
Priority	Medium

Field	Value
Function Name	Edit Profile / Logout
Description	Store manager can update profile information and log out.
Pre-Conditions	Store manager must be logged in.
Priority	Medium

4) Doctor

Field	Value
Function Name	Doctor Login
Description	Allows the doctor to sign in to the dedicated consultation dashboard.
Pre-Conditions	Doctor must have a valid account with the `doctor` role.
Priority	High

Field	Value
Function Name	Answer Customer Questions
Description	Allows the doctor to respond to customer medical questions submitted through the system. Provides medical guidance and advice before customers purchase medicines.
Pre-Conditions	Doctor must be logged in; at least one customer question must be present.
Priority	High

Field	Value
Function Name	Review Patient Medical Records
Description	The doctor can open a patient's stored medical history (allergies, chronic conditions, medications) to inform their answer.
Pre-Conditions	Doctor must be logged in.
Priority	Medium

Field	Value
Function Name	Edit Profile / Logout
Description	Doctor can update profile information and log out.
Pre-Conditions	Doctor must be logged in.
Priority	Medium

3.2.2 Non-Functional Requirements

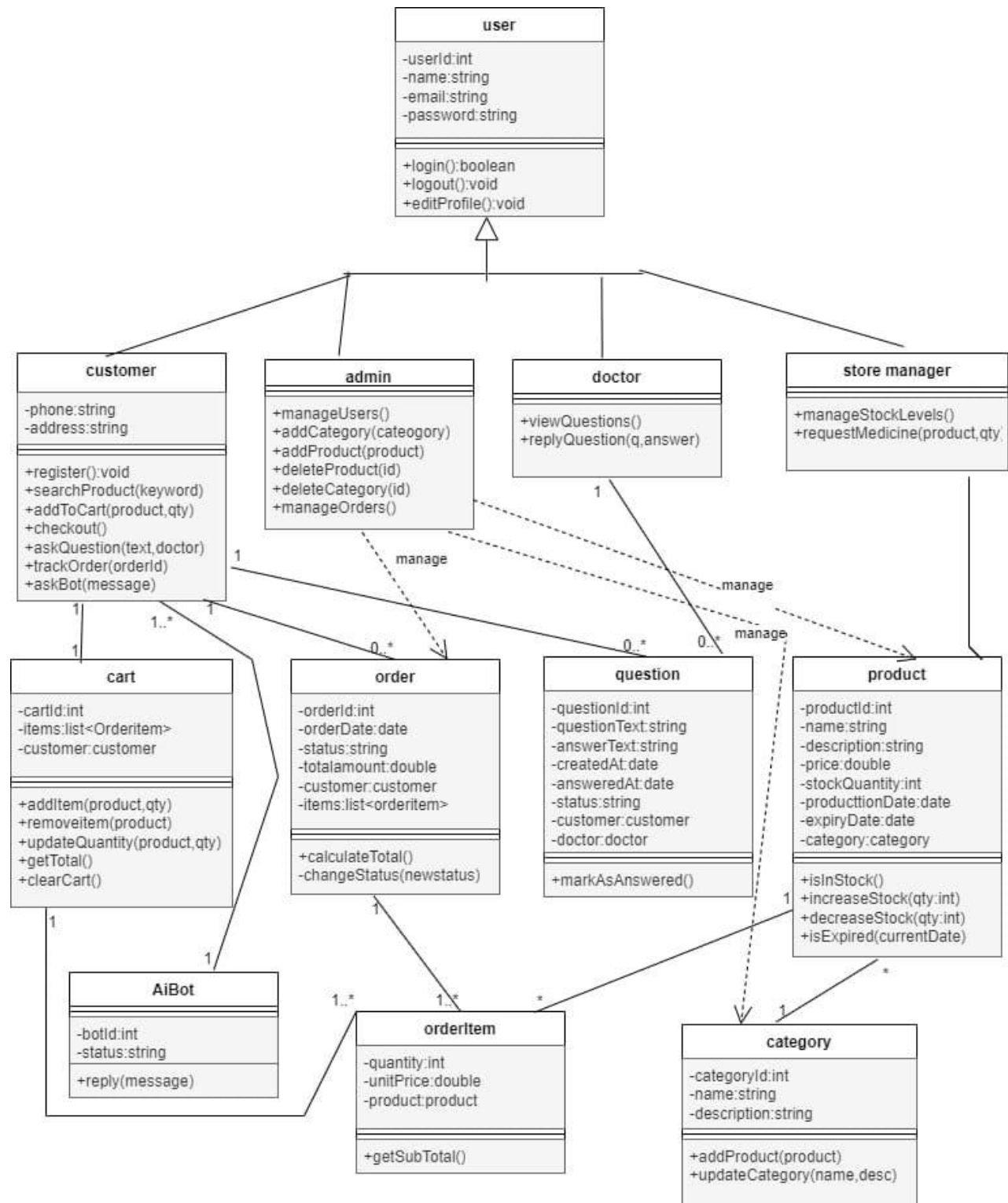
Beyond the behavioral requirements above, the system must satisfy a set of cross-cutting quality attributes. These were used both during design and during testing.

28. **Performance.** The system should respond to user actions (such as searching for medicines, adding items to the cart, and placing orders) within 2 seconds under normal load conditions. The catalog uses a combination of single-page client-side filtering and stale-while-revalidate caching to keep navigation feeling instantaneous after the first load.
29. **Availability.** The platform should maintain an uptime target of 99.9% annually, minimizing downtime caused by maintenance or technical issues. Long-running tasks (such as image uploads) are handled asynchronously where possible to keep request latency low.
30. **Security.** All sensitive data (user passwords and personal information) must be stored securely using cryptographic hashing (Django's PBKDF2-SHA256 by default). Authentication uses short-lived JWT access tokens with refresh tokens; the frontend's HTTP interceptor automatically refreshes expired access tokens without forcing the user to re-login. Permission classes on the backend enforce role boundaries even if the frontend is bypassed.
31. **Usability.** The system should have an intuitive, user-friendly interface that allows users to complete common tasks (such as purchasing medicines or managing products) in a small number of clicks. Visual feedback (toast notifications, loading skeletons, success/error states) is consistent across the application. A global Ctrl+K command palette allows power users to navigate without leaving the keyboard.
32. **Scalability.** The architecture is layered and stateless on the API side, so horizontal scaling is possible by running multiple Django workers behind a load balancer. The database schema is migration-managed and portable to PostgreSQL/MySQL when load increases.

- 33. **Maintainability.** The frontend is built from small, named components with clear prop contracts; complex state lives in dedicated React contexts (Auth, Theme, Wishlist, ProductsCache, Search). The backend follows Django's standard layout (apps, models, serializers, views, urls). Both code-bases use linters and a test suite to catch regressions.
- 34. **Compatibility.** The interface is responsive and validated against viewport widths from 360 px (mobile) to 1920 px (desktop). Modern evergreen browsers (Chrome, Edge, Firefox, Safari) are supported in their last two major versions.
- 35. **Accessibility.** Color contrast meets WCAG AA targets for text, focus rings are visible, dark mode reduces eye strain in low-light environments, and key interactions remain reachable with the keyboard alone (Tab, Enter, Escape, Arrow keys in the command palette and dropdowns).
- 36. **Reliability.** Database operations are wrapped in transactions where multiple rows are written together (e.g., creating an order with its items). The frontend handles network errors gracefully with friendly toast messages instead of raw stack traces.
- 37. **Data Integrity.** Foreign-key relationships and database-level constraints enforce referential integrity; the application layer validates user input before persisting it, preventing inconsistent state such as orders without customers or batches without parent products.

3.3 Software Design

3.3.1 Class Diagram

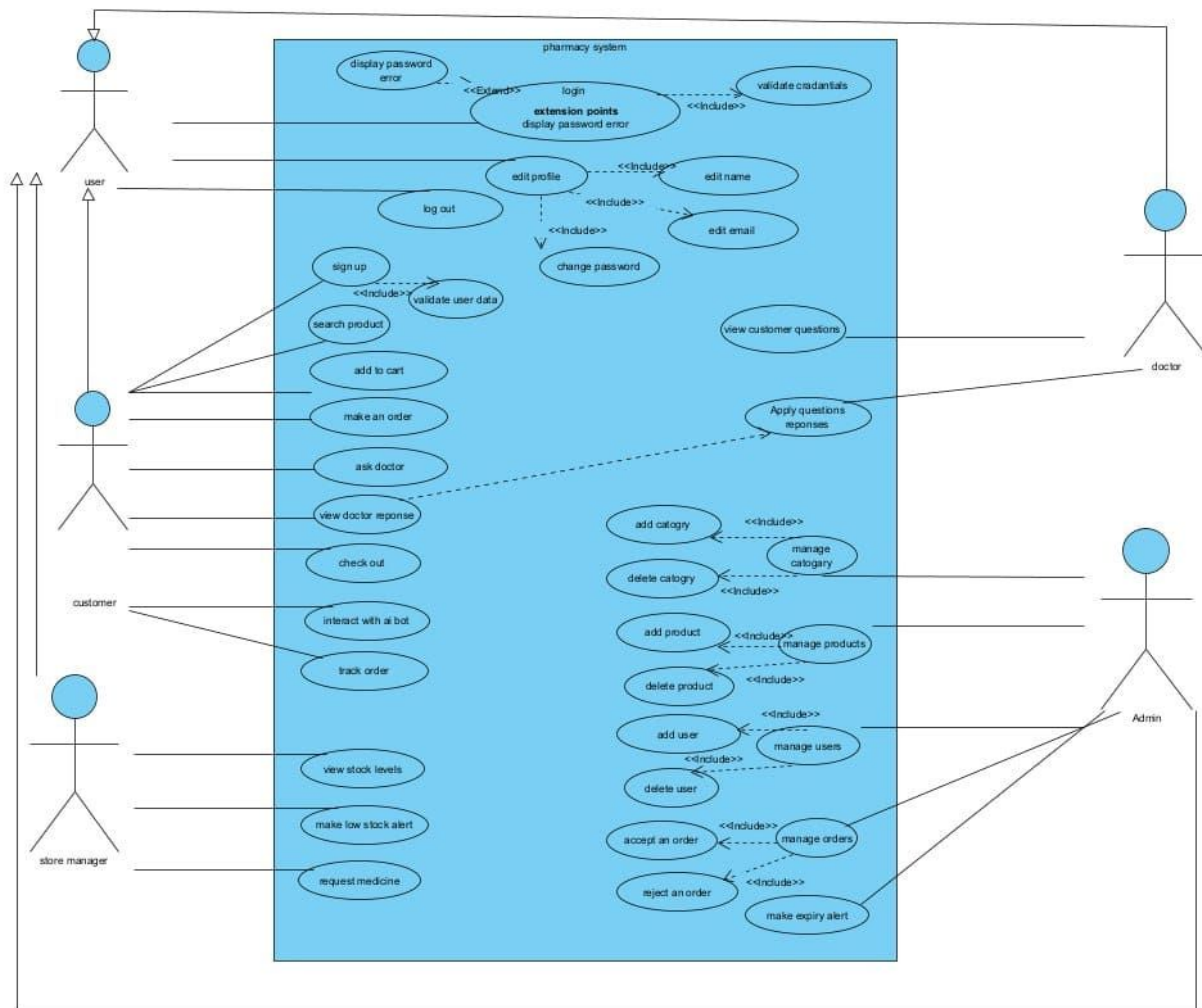


The class diagram captures the principal domain entities and their relationships. The major classes and their responsibilities are summarized below; the full UML diagram appears as a figure in the printed report.

- **User** — Extends Django's `AbstractUser` with additional fields: `role` (one of `customer`, `admin`, `store_manager`, `doctor`), `phone`, and `address`. Hosts methods for authentication via JWT.
- **Category** — A pharmaceutical product category (e.g. Painkillers, Vitamins). Has many Products.
- **Product** — A medicine or healthcare product. Belongs to one Category. Carries the catalog-facing attributes: `name`, `description`, `price`, `manufacturer`, `active_ingredient`, `requires_prescription`, `image_url`, and `low_stock_threshold`. Has many Batches.
- **Batch** — A physical lot of a Product with `batch_number`, `quantity`, `expiry_date`, and `cost_price`. The Product's `total_stock` is computed as the sum of its non-expired batches.
- **CartItem** — An entry in a customer's cart referencing a Product and a quantity. The cart is bound to the User via foreign key.
- **Order** — A submitted purchase by a Customer with `status`, `total_amount`, `payment_method`, `shipping_address`, `created_at`, and `notes`. Has many OrderItems.
- **OrderItem** — A line in an Order referencing a Product, with `quantity` and the historical `price` at the time of purchase.
- **StockRequest** — A replenishment request from a Store Manager to an Admin. References the target Product (and optionally an existing Batch), with `quantity`, `reason`, and `status` (`pending` / `approved` / `rejected`).
- **DoctorQuestion** — A medical question submitted by a Customer to a Doctor with `question_text`, optional `answer_text`, `answered_by` (FK to User with role `doctor`), `created_at`, and `answered_at`.
- **MedicalHistory: Allergy / Condition / Medication** — Three related records on the customer's profile capturing allergens (with severity and reaction), chronic conditions (with status and notes), and current medications (with dosage and frequency).
- **ChatMessage** — A turn in a chatbot conversation between the User and the AI assistant; stores `role` (user/assistant) and `content`.

The diagram makes the principal cardinalities explicit: a User has many Orders, an Order has many OrderItems, a Product has many Batches, a Customer has one MedicalHistory record, and so on. Where bi-directional navigation is helpful (e.g. from Product to its Batches), the model exposes related-name accessors (`product.batches`).

3.3.2 Use Case Diagram and Scenarios



The use case diagram captures the interactions of the four actors—Customer, Admin, Store Manager, Doctor—with the system. The principal scenarios are documented below in goal/actor/precondition/postcondition/process form.

Goal Name	**Sign Up**
Goal	Allow users to create an account in the online pharmacy system.
Actors	Customer.
Pre-Conditions	User is not already registered in the system.
Postconditions	User account is successfully created.
Error Situations	• Missing required information • Invalid input data

	Email or phone already exists
Standard Process	1) User enters personal information. 2) System validates user data. 3) Account is created and a confirmation is shown.

Goal Name	**Login**
Goal	Authenticate users to access the system.
Actors	Customer · Admin · Store Manager · Doctor
Pre-Conditions	User has a valid account.
Postconditions	User is logged in successfully and a JWT pair is issued.
Error Situations	- Invalid credentials - Account not found
Standard Process	1) User enters login credentials. 2) System verifies credentials. 3) System issues access + refresh tokens; user is routed to their role-specific landing screen.

Goal Name	**Logout**
Goal	Allow users to securely exit the system.
Actors	Customer · Admin · Store Manager · Doctor
Pre-Conditions	User is logged in.
Postconditions	User session is terminated; tokens are cleared client-side.
Error Situations	Session termination fails.
Standard Process	1) User triggers logout. 2) System clears tokens and redirects to the login screen.

Goal Name	**Search Product**
Goal	Allow users to search for products in the online pharmacy system.
Actors	Customer.
Pre-Conditions	Products are available in the system.
Postconditions	Matching products are displayed to the user.
Error Situations	No products found.
Standard Process	1) User enters product name, keyword, or category. 2) System searches the product database (name, manufacturer, active ingredient, description). 3) Matching products are displayed.

Goal Name	**Manage Products**
------------------	----------------------------

Goal	Allow the admin to manage products in the online pharmacy system.
Actors	Admin.
Pre-Conditions	Admin is logged in with valid credentials.
Postconditions	Product information is successfully added, updated, or deleted.
Error Situations	• Invalid product data • Missing required information • Product not found
Standard Process	1) Admin selects the product management option. 2) Admin adds a new product or updates / deletes an existing one. 3) System validates the data. 4) Product data is saved and the product list is updated.

Goal Name	**Manage Users**
Goal	Allow the admin to manage Users in the online pharmacy system.
Actors	Admin.
Pre-Conditions	Admin is logged in with valid credentials.
Postconditions	User accounts are successfully viewed, updated, or deleted.
Error Situations	• User not found • Invalid user data • Unauthorized action
Standard Process	1) Admin selects the user management option. 2) Admin views the list of users. 3) Admin updates user information or deletes user accounts when required. 4) System saves changes and updates user records.

Goal Name	**Manage Order**
Goal	Allow the customer to place an order for selected products.
Actors	Customer.
Pre-Conditions	Customer is logged in and the shopping cart is not empty.
Postconditions	Order is successfully created and submitted for processing.
Error Situations	• Cart is empty • Product out of stock • Order creation failed
Standard Process	1) Customer reviews the products in the shopping cart.

	2) Customer confirms the order with shipping address and payment method. 3) System checks product availability across non-expired batches. 4) System creates the order, deducts stock, and updates its status to Pending.
--	--

Goal Name	**Add to Cart with Allergy Check**
Goal	Add a product to the customer's cart while warning about known allergies.
Actors	Customer.
Pre-Conditions	Customer is logged in; product exists in the catalog.
Postconditions	Product is added to the cart, or a clear warning is shown and the customer is given the choice to proceed.
Error Situations	• Product out of stock • Cart-add API fails
Standard Process	1) Customer clicks "Add to Cart" on a product. 2) System checks the product's active ingredient against the customer's recorded allergies. 3) If a match is found, an allergy warning is displayed with a confirm/cancel choice. 4) On confirm, the product is added to the cart and a success toast is shown.

Goal Name	**Make Low Stock Alarm**
Goal	Notify the store manager when product stock reaches a low level.
Actors	Store Manager.
Pre-Conditions	Store manager is logged in and products exist in the system.
Postconditions	Low-stock notification is generated and displayed to the store manager.
Error Situations	• Product data not available • Notification generation failed
Standard Process	1) System monitors product stock levels. 2) System detects products whose total non-expired stock has fallen below the configured threshold. 3) System generates a low-stock indicator on the manager's dashboard. 4) Store manager reviews the alerts and may submit a stock request.

Goal Name	**Submit Stock Request**
Goal	Allow the store manager to request replenishment for a low-stock product.
Actors	Store Manager.
Pre-Conditions	Store manager is logged in; the target product exists.
Postconditions	A pending stock request is created and visible to the admin.
Error Situations	- Quantity invalid (≤ 0) - No reason provided
Standard Process	1) Manager opens the request modal for a product. 2) Manager fills quantity, reason, and either selects an existing batch or supplies a new batch number with expiry. 3) System validates and stores the request with status 'pending'.

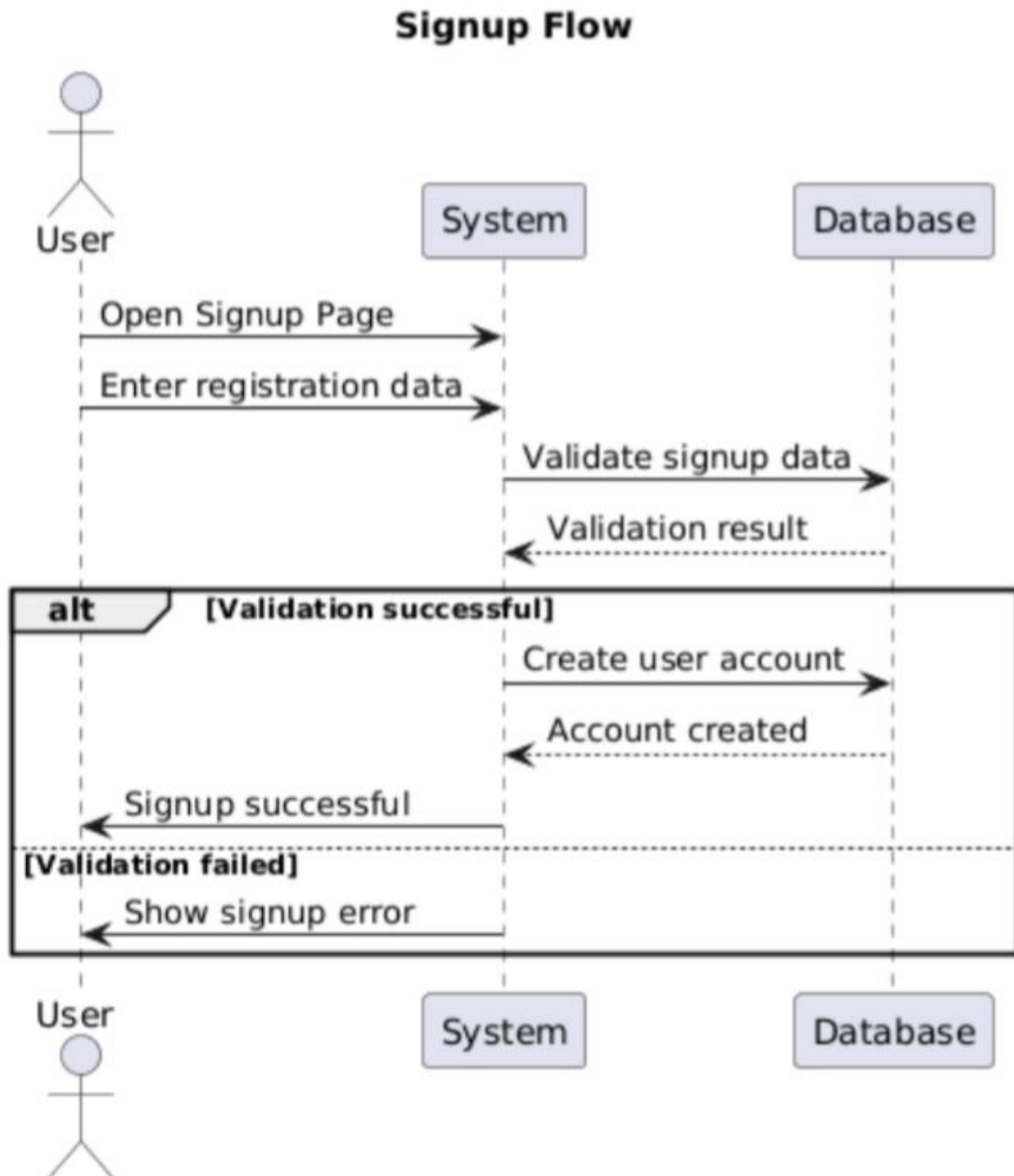
Goal Name	**Approve Stock Request**
Goal	Allow the admin to approve a pending stock request.
Actors	Admin.
Pre-Conditions	Admin is logged in; at least one pending stock request exists.
Postconditions	The request is marked approved and stock is added to the relevant batch.
Error Situations	- Stock request not found - Concurrency conflict
Standard Process	1) Admin reviews the request. 2) Admin approves; the system either updates the existing batch quantity or creates a new batch as specified. 3) Manager sees the request status flip to approved.

Goal Name	**Apply Question Response**
Goal	Allow the doctor to respond to customer questions through the system.
Actors	Doctor.
Pre-Conditions	Doctor is logged in and customer questions are available.
Postconditions	Doctor response is successfully sent to the customer.
Error Situations	- Question not found - Response submission failed
Standard Process	1) Doctor views the list of customer questions.

	2) Doctor selects a question to respond to and (optionally) reviews the patient's medical history. 3) Doctor writes and submits the response. 4) System delivers the response to the customer.
--	--

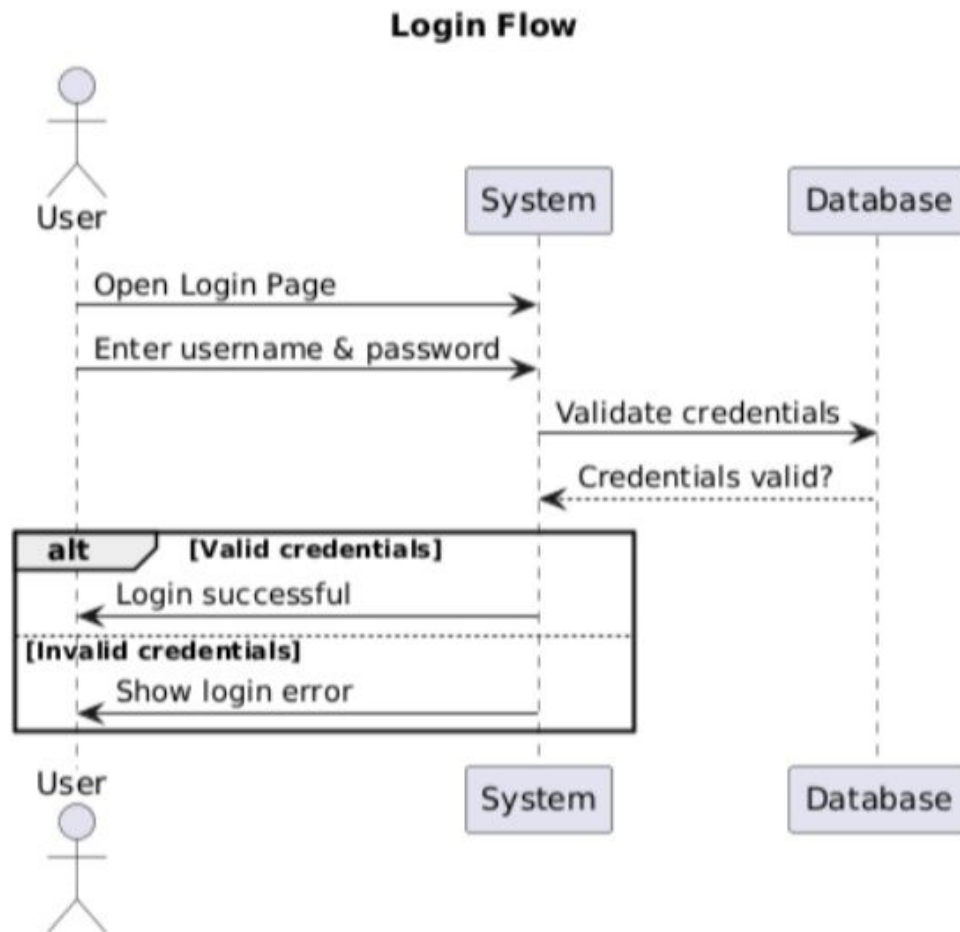
3.3.3 Sequence Diagram

The sequence diagrams document the time-ordered exchange of messages between the actors, the React frontend, the Django REST API, and the SQLite database for the most representative flows.



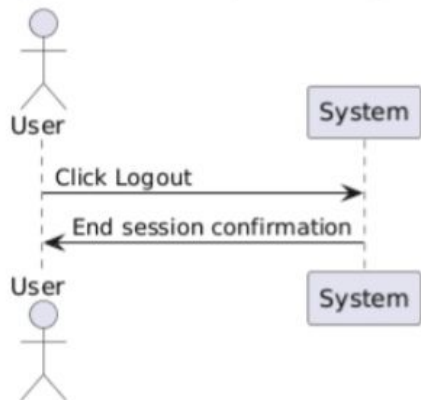
- **Sign Up.** The user fills the registration form and submits it. The frontend POSTs the form data to ``/api/users/register/'`. The backend validates the payload, hashes the password, persists a User row,

and returns a success response with the new user's basic profile. The frontend redirects to the login screen.

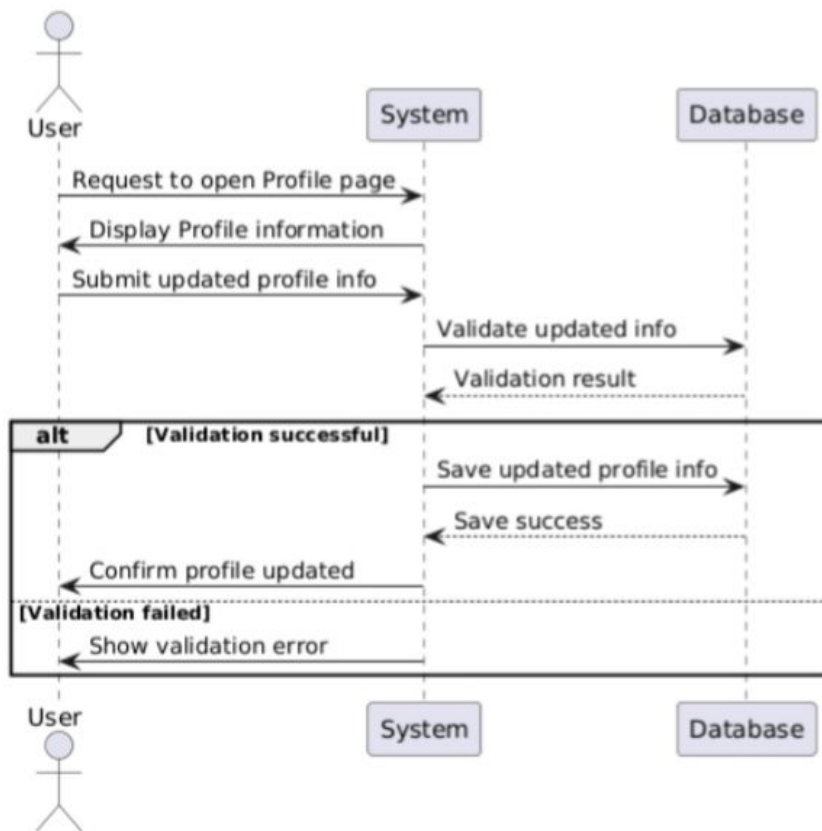


- **Login.** The user submits credentials. The frontend POSTs to `/api/auth/login/`. SimpleJWT verifies the password against the hashed value, issues an access token (short TTL) and a refresh token (longer TTL), and returns them. The frontend stores both, decodes the role from the access token, and routes the user to their role-specific landing page.
- **Browse and Order.** The customer browses the catalog (GET `/api/products/?paginate=false`), adds items to the cart (POST `/api/cart/add/`), and finally submits the order (POST `/api/orders/`). The order endpoint creates the order, creates one OrderItem per cart row, decrements stock, and clears the cart in a single transaction.

Logout Flow - Sequence Diagram

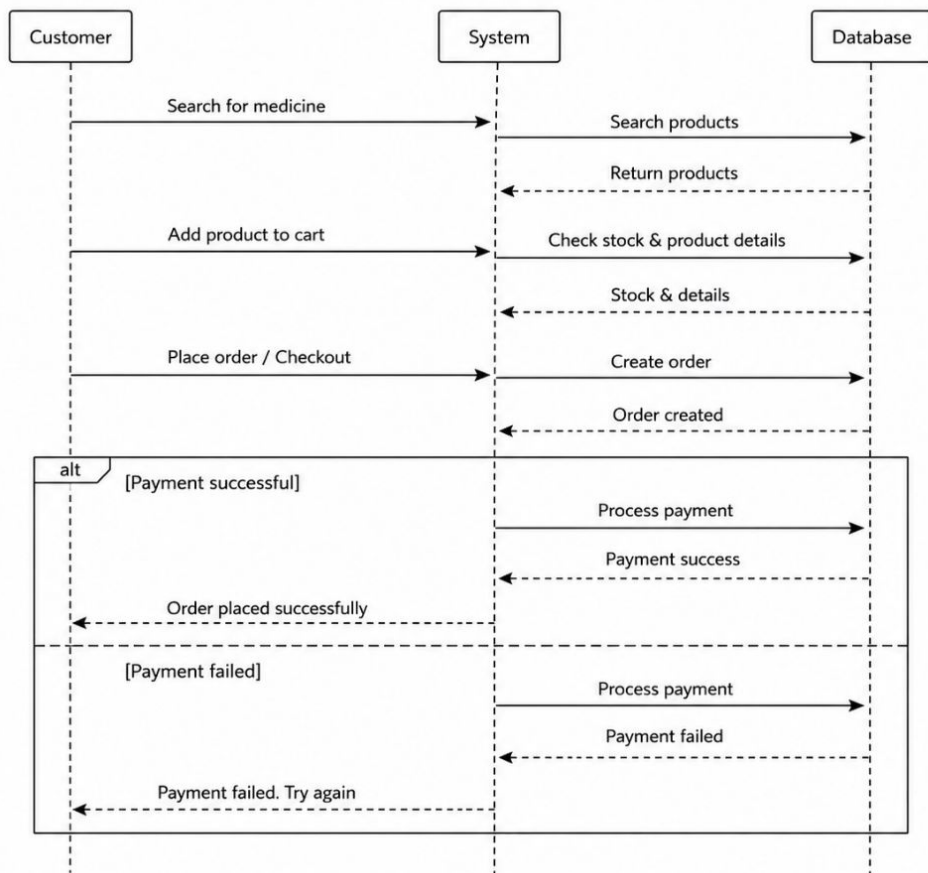


Edit Profile Flow



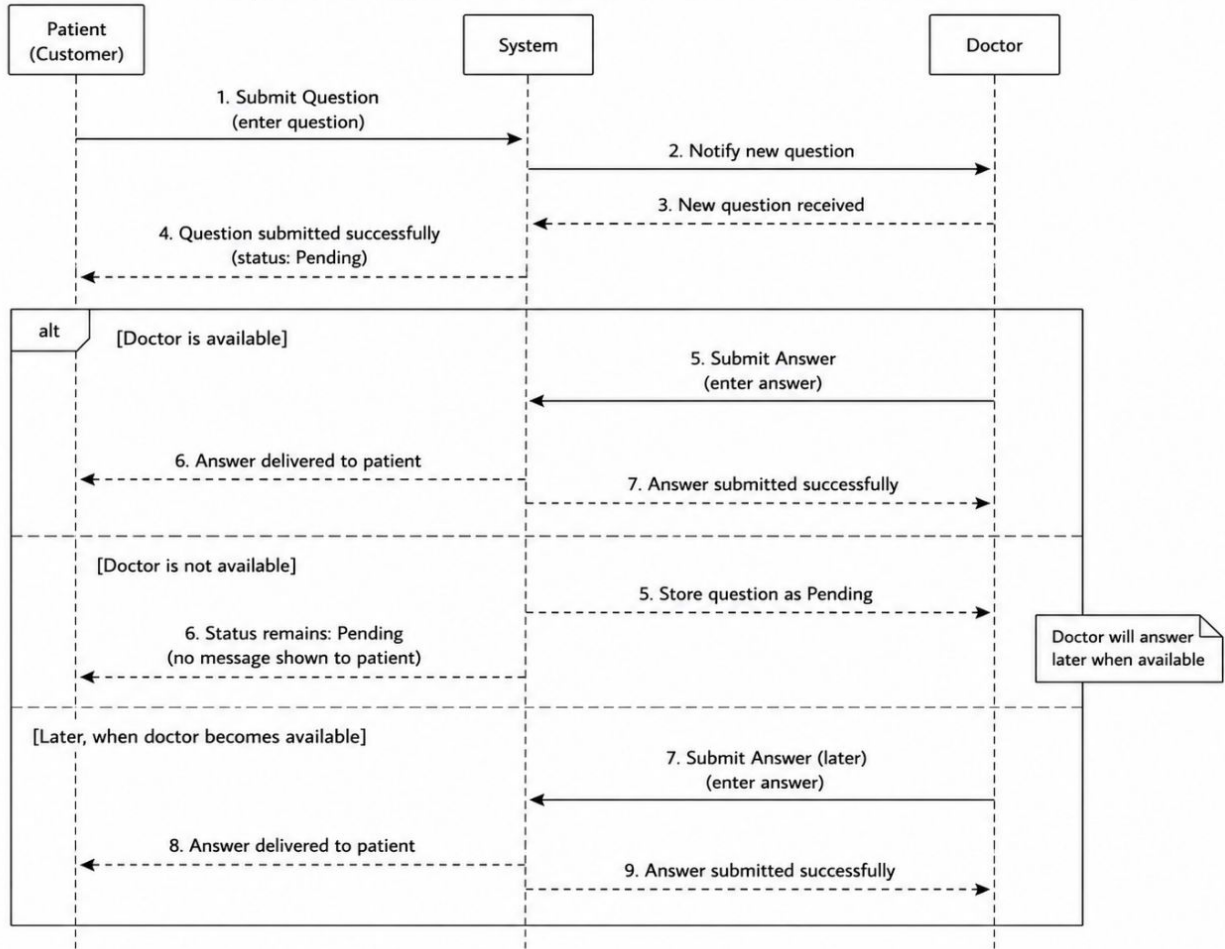
- **Logout.** which validates and persists the change. Logout simply clears tokens client-side and routes the user to the login screen.

Sequence Diagram – Purchase (Place Order)



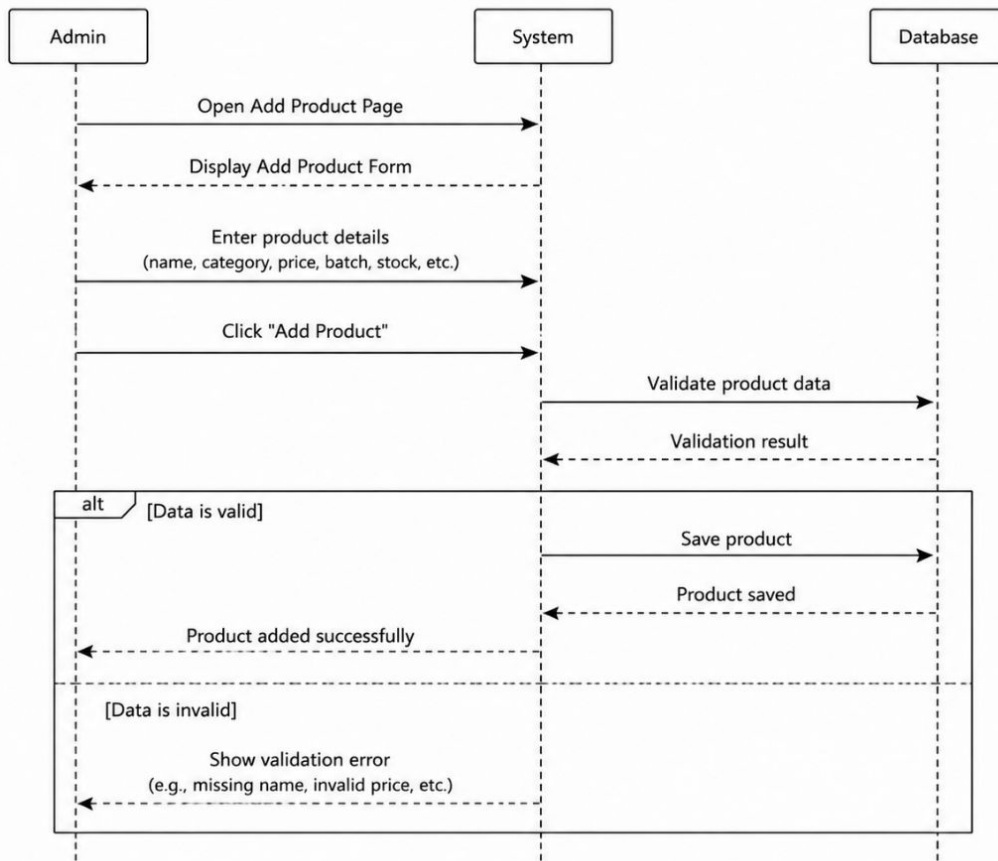
Purchase Sequence Diagram: This diagram describes the process of searching for medicine, adding products to the cart, placing an order, and completing the payment through the pharmacy system.

Sequence Diagram – Doctor Answers Patient Questions

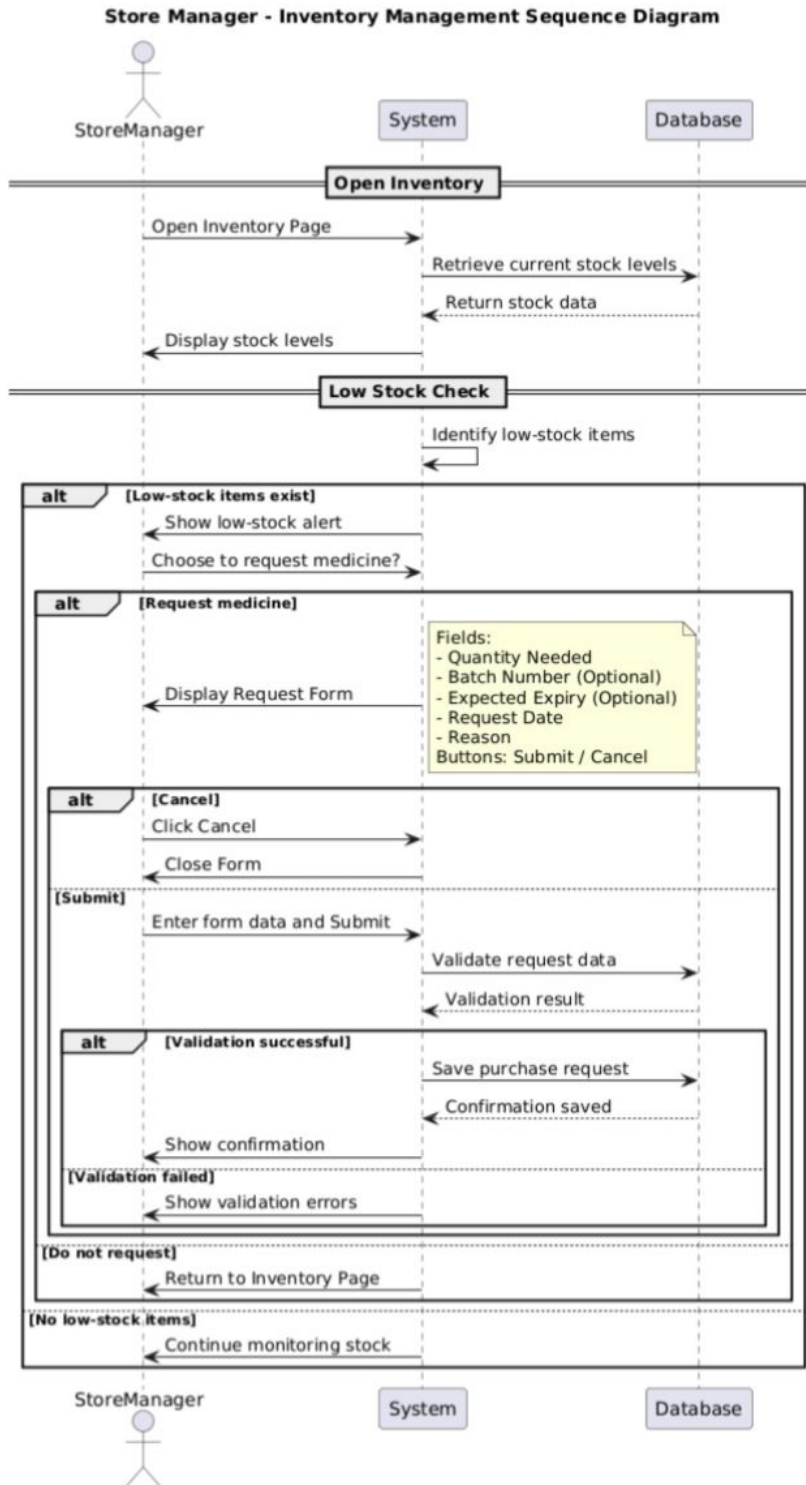


Doctor Answers Patient Questions Sequence Diagram: This diagram shows how patients submit questions to doctors and how doctors respond when available, while unanswered questions remain pending until later response.

Sequence Diagram – Admin Add Product

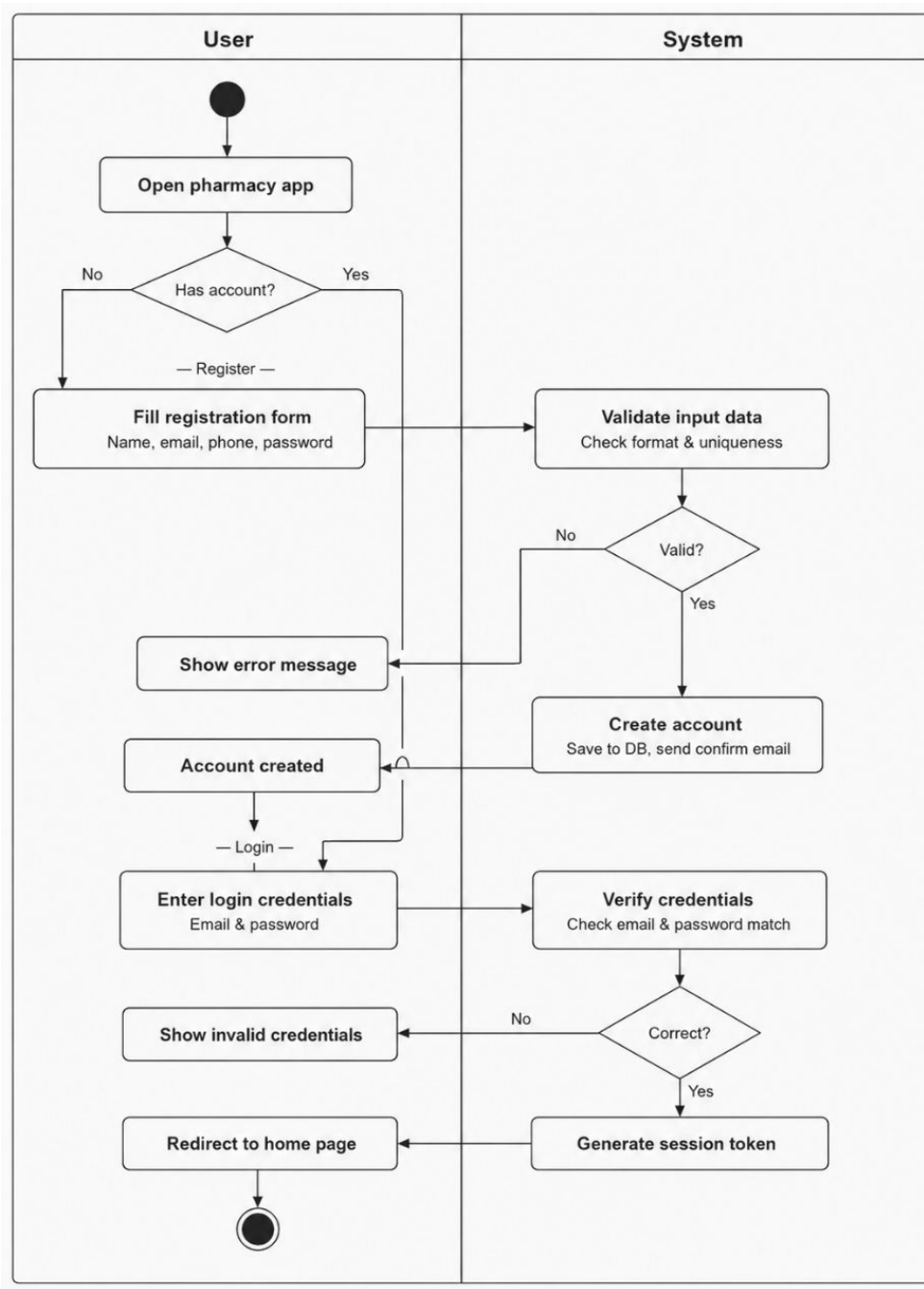


Admin Add Product Sequence Diagram: This diagram explains how the admin adds a new product to the system, validates product data, and stores it in the database.

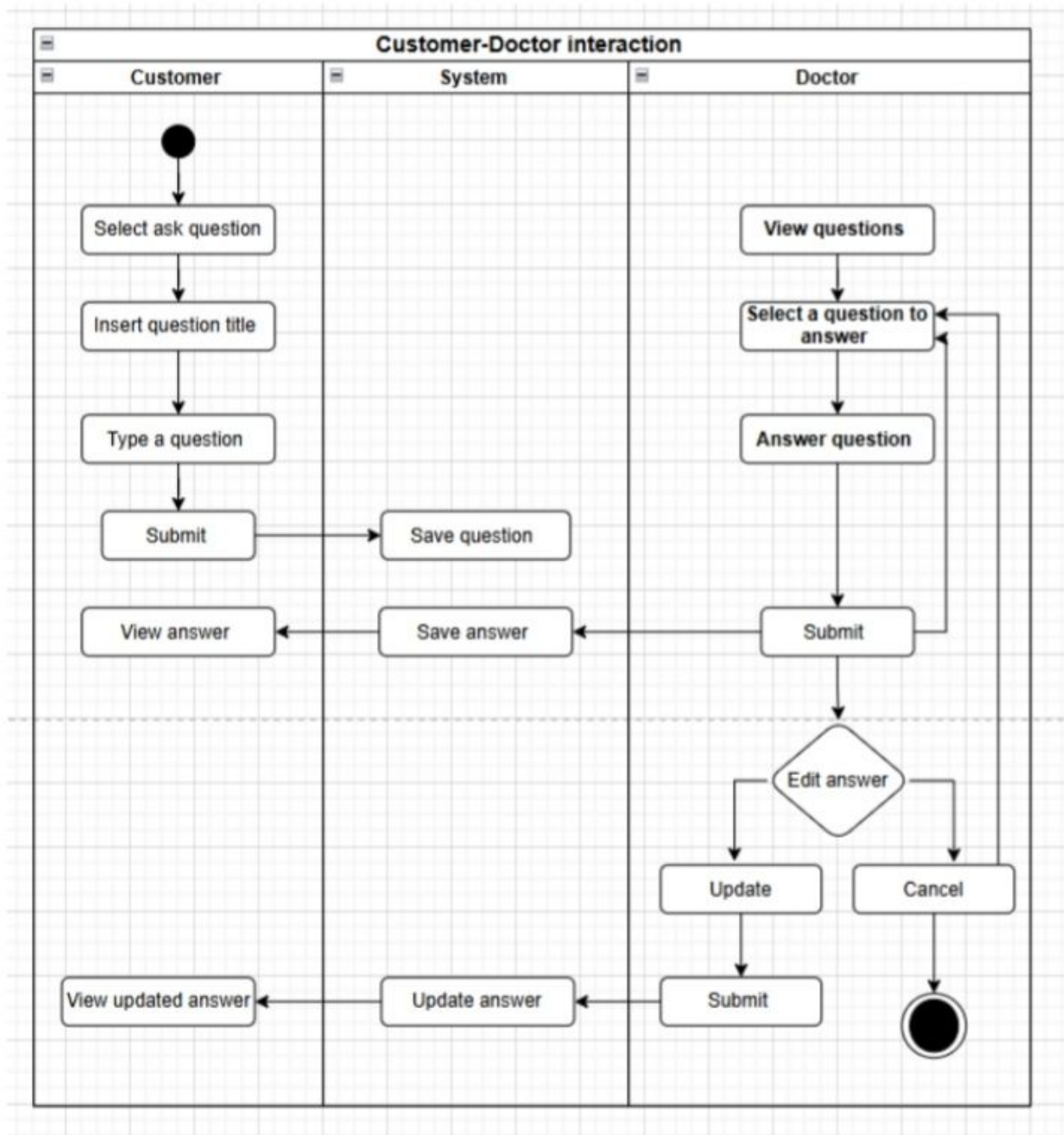


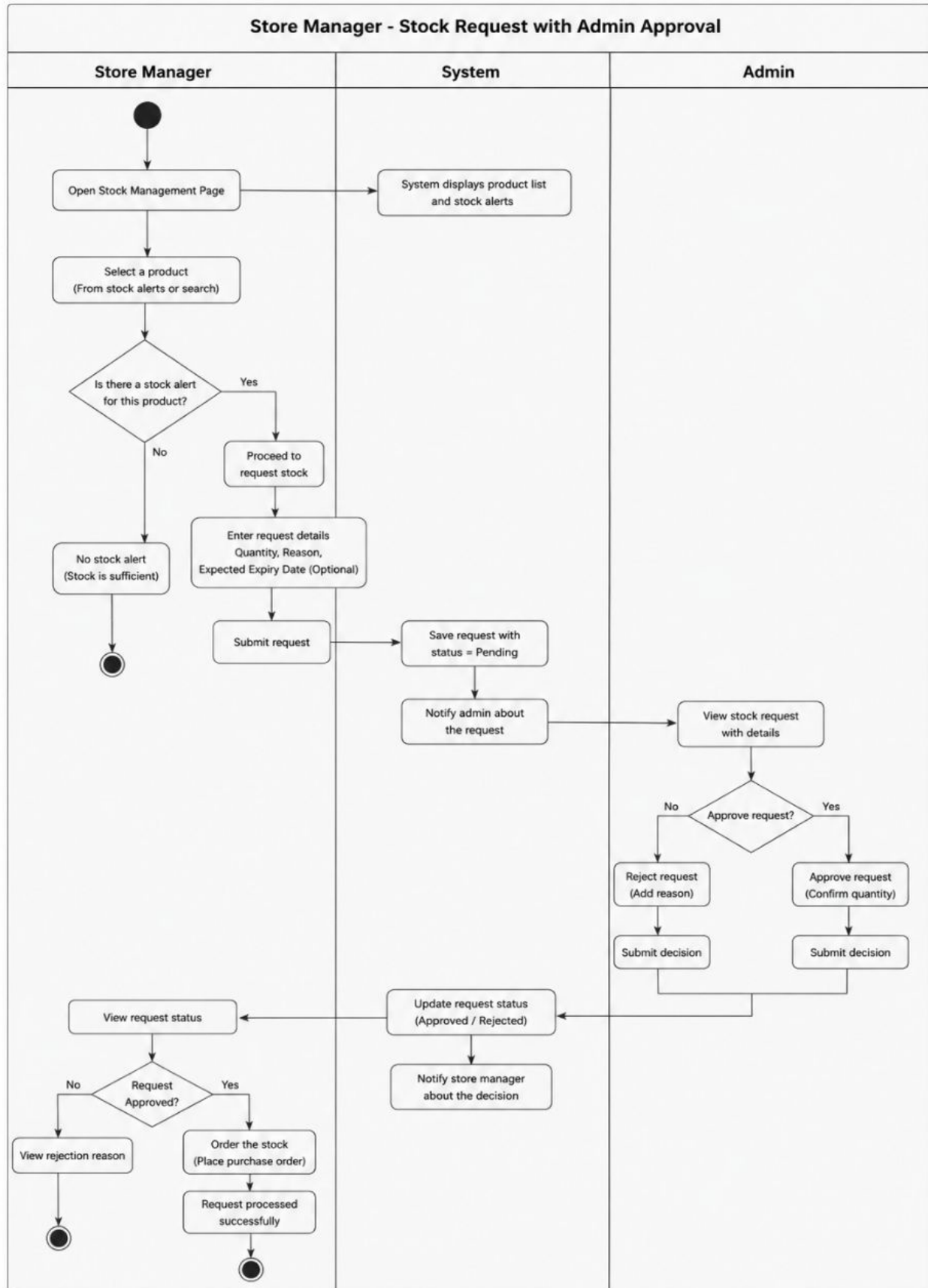
Store Manager Sequence Diagram: This diagram illustrates how the store manager monitors product stock levels and requests missing or low-stock medicines to maintain inventory availability.

3.3.4 Activity Diagram

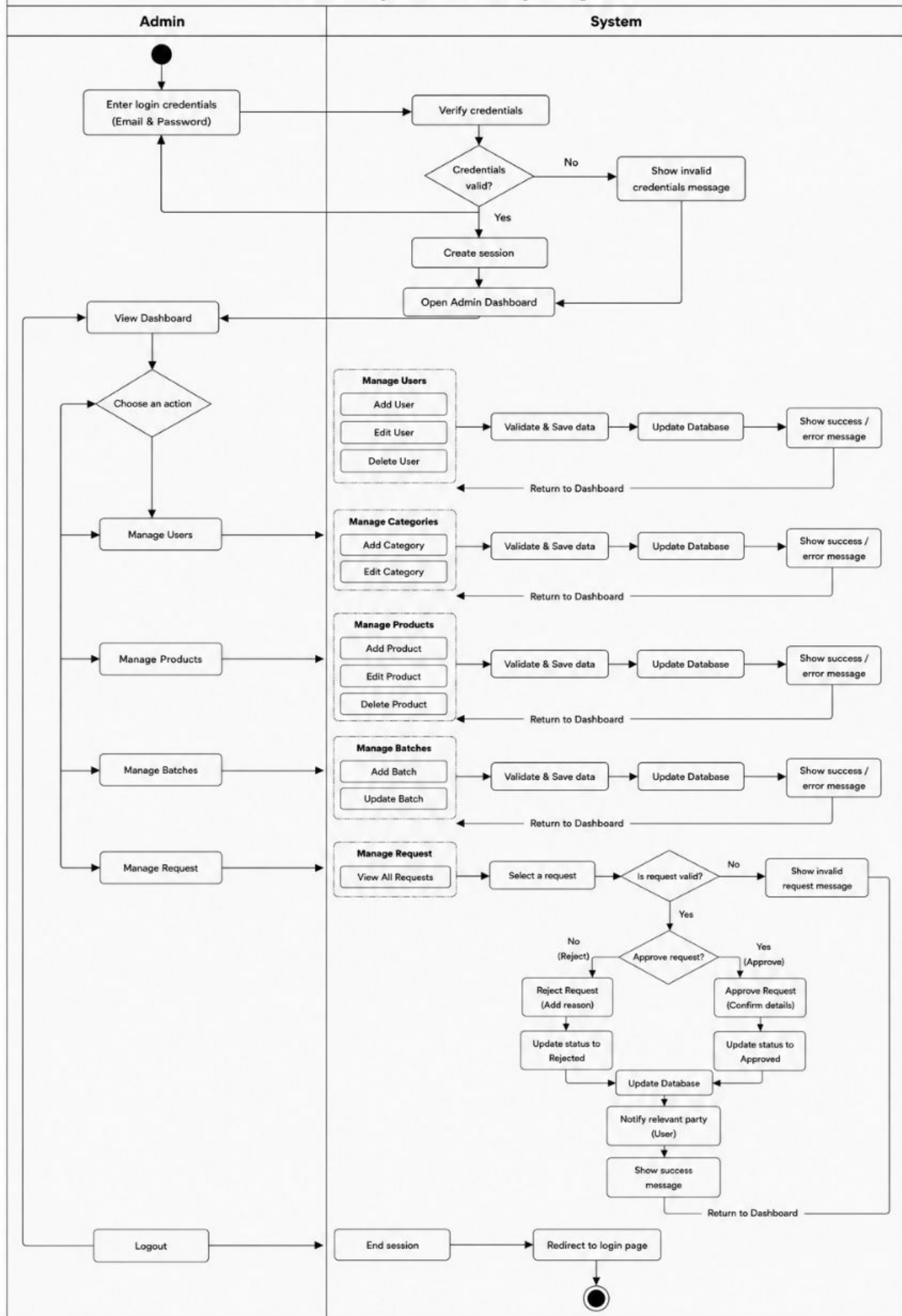








Admin - System Activity Diagram



The activity diagrams describe the control flow of the major workflows from the user's point of view, including decision points (e.g. "is this user authenticated?"), parallel activities, and terminal states.

The principal activities documented are:

- **Sign Up.** Captures the form-validation branches and the success/failure terminal states.
- **Login.** Captures the credential check, the role decoding, and the role-specific routing branch.
- **Logout.** Captures the simple termination flow with token cleanup.
- **User Account.** Captures the profile-update flow including validation and confirmation.
- **Customer & System.** Captures the end-to-end purchase journey from browse → cart → checkout → order tracking, including the allergy-check branch on cart-add.
- **Customer & Doctor.** Captures the question-submission and reply-receipt cycle.
- **Store Manager & System.** Captures the low-stock detection, request submission, and request status update flow.
- **Admin & System.** Captures the master administrative activities: managing categories, products, users, orders, and approving stock requests.

Each diagram is rendered as a UML activity diagram with start/end nodes, decision diamonds, and synchronization bars where applicable.

CHAPTER 4 — IMPLEMENTATION

4.1 Software Architecture

The Online Pharmacy System follows a modern three-tier architectural pattern, consisting of a presentation layer (frontend), a business logic layer (backend), and a data access layer (database). This architecture ensures a clear separation of concerns, making the system easier to maintain, secure, and scale as the number of users and products grows.

The presentation layer is responsible for user interaction; the business logic layer handles application rules and workflows; and the data access layer manages data storage and retrieval operations. The three layers communicate via HTTP using a REST API for the client–server boundary and the Django ORM for the application–database boundary.

Client-Side Architecture

The client side of the Online Pharmacy System is developed using **React.js (version 19)**, which provides a component-based architecture for building dynamic and responsive user interfaces. The frontend application runs in modern web browsers and communicates with the backend server through RESTful APIs over JSON.

The client-side architecture focuses on providing an easy-to-use interface for customers, administrators, store managers, and doctors while ensuring fast navigation and a smooth user experience. Each role has its own distinct dashboard and navigation, but all share a common visual system based on Tailwind CSS and a small library of reusable components (buttons, cards, modals, toasts, confirm dialogs).

User Interface Components

The frontend consists of reusable React components that handle different system features, including:

- **Catalog and search:** ``Products``, ``ProductCard``, ``ProductSkeleton``, ``QuickPreviewModal``, ``SortDropdown``, ``SearchPalette``, ``FeaturedProducts``, ``CategoriesShowcase``.
- **Cart and checkout:** ``Cart``, ``CartContext``, allergy-warning modal.
- **Authentication flows:** ``Login``, ``Signup``, ``PrivateRoute``, ``AuthContext``.
- **Customer dashboard:** ``Home`` (role-aware), ``Orders``, ``Profile``, ``MedicalHistory``, ``AskDoctor``, ``Chatbot``.
- **Admin dashboard:** ``ManageUsers``, ``ManageCategories``, ``ManageProducts``, ``ManageBatches``, ``ManageOrders``, ``ManageStockRequests``.
- **Store manager interface:** ``StockManagement`` with low-stock dashboard, batch-view modal, and stock-request workflow.
- **Doctor interface:** ``DoctorQuestions`` (Q&A inbox), ``PatientMedicalRecords``.

- **Layout and chrome:** ``Navbar`` (role-aware), ``Footer``, ``Hero``, ``Testimonials``, ``TrustedBrands``, ``ScrollToTop``, ``ThemeToggle``, ``HashScrollHandler``.

These components keep the user interface organized, support code splitting (each page is loaded lazily on first navigation), and are easy to maintain in isolation.

State Management

State management in the Online Pharmacy System is handled by a combination of React's primitive hooks (``useState``, ``useEffect``, ``useMemo``, ``useCallback``, ``useDeferredValue``) and a small set of React Contexts that own cross-cutting application state:

- **``AuthContext``** — Holds the authenticated user, exposes ``login``, ``logout``, and ``register`` actions, and centralizes JWT storage and refresh.
- **``ThemeContext``** — Tracks dark/light mode, persists the choice to ``localStorage``, and follows the system preference when no explicit choice has been made.
- **``WishlistContext``** — Holds the wishlist for the current user, persisted under a per-user key (``pharmacare:wishlist:user-<id>``) and synchronized across browser tabs via the ``storage`` event.
- **``ProductsCacheContext``, ``CategoriesCacheContext``, ``BatchesCacheContext``** — Three sibling resource caches that each wrap a single GET endpoint with stale-while-revalidate semantics so that any consumer (storefront, search palette, admin pages, manager dashboard) reads the data instantly after the first load. See the next subsection for details.
- **``SearchContext``** — Owns the open/close state of the global Ctrl+K palette and registers the global keyboard listener.

This division keeps each component focused on its own concerns while making genuinely shared state available without prop drilling.

Resource Caching Architecture

A recurring source of perceived slowness in early iterations was that each admin or manager page fetched its own copy of the same lists on every mount. Navigating between **Manage Products** → **Manage Batches** → **Manage Categories** triggered three full network round-trips even though the underlying data rarely changes during a session, and switching the filter chip on the batches page (``All / Expired / Expiring Soon``) called a *different* endpoint every click. The customer-facing storefront and the global Ctrl+K palette suffered from the same duplication.

To eliminate this, all read-heavy GET resources were placed behind a small, generic stale-while-revalidate cache built specifically for this project. The factory function ``createResourceCache(...)`` returns a ``(Provider, useCache)`` pair for any single-endpoint resource and provides the following guarantees:

- **Instant reads after warm-up.** Once the cache holds data, every consumer reads from memory; the network is not contacted at all unless the data is older than ``STALE_MS`` (60 s).

- **In-flight coalescing.** Concurrent calls to ``refetch()`` are deduplicated through an in-flight ref, so multiple components mounting at the same time share a single network request.
- **Background revalidation.** A stale cache *still* renders immediately. The fresh request runs in the background, and the loading flag flips to ``true`` only when the cache is genuinely empty — so a background refresh never replaces rendered rows with a spinner.
- **Idle prefetch on app boot.** The provider schedules a one-shot fetch via ``requestIdleCallback`` so that the cache is usually warm before the user clicks anything.
- **Explicit ``invalidate()`` on writes.** Mutations (create/update/delete) call ``invalidate()`` to mark the cache dirty; the next ``refetch()`` then bypasses the freshness check and goes to the network. The cache deliberately does not auto-invalidate on writes the consumer didn't tell it about, so divergence is impossible to silently introduce.
- **``byId`` lookup map.** Each cache exposes a memoized ``Map(String(id) → item)`` so consumers like the search palette's focus-on-arrival flow can look up an entity in $O(1)$ without re-scanning the array.

Three providers are mounted at the application shell (``ProductsCacheProvider``, ``CategoriesCacheProvider``, ``BatchesCacheProvider``) and consumed by the relevant pages through their ``useProductsCache()``, ``useCategoriesCache()``, and ``useBatchesCache()`` hooks. Because the providers live above the routing layer, navigating between admin / manager / customer screens never tears down the cache.

A particularly impactful follow-on optimization was made possible by this architecture: the **Manage Batches** page no longer calls three separate endpoints when the user clicks between filter chips. Instead, ``getAll()`` is fetched once into the cache, and the page derives the *expired*, *expiring soon*, and *valid* subsets in memory using the ``is_expired`` and ``expiry_date`` fields the API already returns. A single pass through the cached array also computes the four header stats (total / expired / expiring / valid) and the search-filtered subset in one walk, replacing five separate ``filter()`` calls on every keystroke.

Render-Path Optimizations

Beyond the cache layer, the heavier admin and manager pages apply a small set of standard React optimizations whose combined effect is to keep input devices feeling responsive even when the catalog grows to several hundred entries:

- **``useMemo`` for derived data.** All filtered, sorted, and aggregated arrays (``filteredProducts``, ``filteredBatches``, ``lowStockProducts``, header stats) are memoized so they recompute only when their actual inputs change — not on every modal toggle, hover, or unrelated state update.
- **``useDeferredValue`` for search inputs.** Typing in a search field updates the local state immediately (so the input stays smooth at 60 fps) but the filtering work runs against the deferred value, catching up on the next idle frame. This decouples keystroke latency from list size.
- **``React.memo`` on row/card components.** Extracted card components such as the categories grid item are wrapped in ``React.memo`` so a search-bar keystroke that only changes the parent's ``searchTerm``

state does not re-render every visible card. To make memoization actually pay off, all callback props passed into memoized children (`onEdit`, `onDelete`) are stabilized with `useCallback` so their identity does not change between renders.

- **Skeleton placeholders instead of spinners.** While the cache is cold, a layout-matching skeleton (`TableSkeleton`, `CardGridSkeleton`) is rendered instead of a single centered spinner. The skeleton has the same column count and approximate row height as the real data, so the page does not reflow when the data arrives — which significantly improves perceived performance.
- **Code splitting with idle prefetch.** Each top-level page is loaded via `React.lazy` and rendered behind `<Suspense>`; an idle prefetcher warms the most-likely-next route in the background, so the *first* navigation to a heavy admin page already has the JS chunk in cache.

Routing

Client-side routing is implemented with **React Router 6**. Routes are organized so that customers, administrators, store managers, and doctors land on their own dashboard after logging in. A `PrivateRoute` wrapper enforces that protected routes redirect unauthenticated users to the login screen, and that role-restricted routes (e.g. `/admin/*`) bounce users with insufficient privileges back to a safe default.

To keep the initial bundle small, each major page is loaded with `React.lazy` and rendered behind `<Suspense>` with a skeleton fallback. An idle prefetcher (using `requestIdleCallback`) warms up the most-likely-next routes while the browser is otherwise idle, so subsequent navigation feels instantaneous.

HTTP Client

The **Axios** library is used in the React frontend to send HTTP requests to the Django backend APIs. A pair of interceptors centralize cross-cutting concerns:

- The **request interceptor** attaches the JWT access token to outbound requests when available.
- The **response interceptor** detects HTTP `401` responses, automatically calls the refresh endpoint to obtain a new access token, and replays the original request transparently. If the refresh itself fails, the user is redirected to the login screen.

This pattern means the rest of the application can call API endpoints without thinking about token management.

Server-Side Architecture

The backend of the Online Pharmacy System is implemented using **Django 4.2+** together with **Django REST Framework (DRF)**, providing a scalable and secure RESTful API. The server-side architecture is responsible for handling business logic, processing requests, and managing communication with the database.

The server-side architecture includes:

API Views and Controllers

Handle HTTP requests and responses, implementing business logic for:

- User management (register, login, refresh, profile, logout).
- Medicine catalog management (CRUD on categories, products, batches).
- Cart and order processing (cart add/remove/update, checkout, order status transitions).
- Doctor consultation Q&A (submit question, list questions, post answer).
- AI chatbot integration (forward message to OpenRouter, persist conversation turn).
- Stock request workflow (manager submit, admin approve/reject).
- Medical history records (allergies, conditions, medications).

These are implemented as DRF `ViewSet`'s and function-based views where the resource model is straightforward, and as plain `APIView`'s for endpoints with bespoke behavior (e.g. `/api/cart/check-allergy/`).

Business Logic Layer

Contains the core application logic, validation rules, prescription handling, stock validation, and workflow management for orders and approvals. Examples include:

- **Allergy checker:** when a customer adds a product to the cart, the system inspects the customer's stored allergies and the product's `active_ingredient` and surfaces a warning if a match is found.
- **Stock decrement:** when an order is placed, stock is deducted across the product's non-expired batches in expiry order (first-expiry-first-out).
- **Order status state machine:** explicit allowed transitions (Pending → Approved → Processing → Shipped → Delivered, with Cancelled / Rejected as terminal alternatives).
- **Stock-request approval:** approving a request either tops up an existing batch's quantity or creates a new batch with the supplied number and expiry date.
- **Expired product handling:** when a product becomes expired, the system automatically removes it from the customer product listing to prevent users from purchasing unavailable or unsafe medicines.

Data Access Layer

The **Django ORM** manages database interactions, allowing efficient querying, updating, and relationship handling between system entities such as users, medicines, categories, batches, orders, and stock requests. Migrations are used to evolve the schema in a tracked, repeatable way; querysets keep most logic declarative; and the admin scaffolding gives a useful quick view of the data during development.

Authentication & Authorization

Token-based authentication is used to ensure secure user access. SimpleJWT issues a pair of tokens at login: a short-lived **access token** (used to authenticate each API call) and a longer-lived **refresh token**

(used to obtain a new access token without prompting the user to log in again). DRF's permission classes are layered so that:

- `IsAuthenticated` is the default for protected endpoints.
- A custom `IsRole` permission accepts one or more role names and rejects requests from authenticated users who don't hold the right role.
- A few endpoints (catalog read, login, registration) are explicitly exposed via `AllowAny` so the storefront works for unauthenticated visitors.

This combination ensures that even if the frontend is bypassed (for example, by hand-crafting requests), the backend continues to enforce role boundaries.

Code Organization

The repository is structured into two top-level applications:

```
`` backend/ pharmacy_project/      # Django project (settings, urls, wsgi) pharmacy/          # Domain
app (models, views, serializers, urls) manage.py requirements.txt

frontend/ src/ components/         # Reusable UI primitives + role-aware navbar/footer pages/
# Top-level routed screens (one folder per role) contexts/           # Auth, Theme, Wishlist,
ProductsCache, Search hooks/      # Custom hooks (toast, focus-on-arrival, cart count) services/
# Axios client + per-resource API wrappers lib/                      # Pure helpers (search providers, etc.) public/
# Static assets (images, favicons) package.json tailwind.config.js ``
```

This organization mirrors the layered architecture described above and keeps the domain logic separated from the presentation logic.

4.2 Database Architecture

The system uses an **SQLite** database to store application data with properly structured tables and relationships to ensure consistency and maintainability. The schema is migration-driven and is portable to PostgreSQL or MySQL for production deployment without changes to the application code.

Principal Tables

User Management

Stores user accounts including admins, store managers, doctors, and customers, with role-based access permissions. Built on top of Django's `AbstractUser` and extended with the additional fields needed by this application.

Column	Type	Notes
id	INT (PK)	Auto-incremented identifier
username	VARCHAR	Unique

email	VARCHAR	Unique
password	VARCHAR	Hashed (PBKDF2-SHA256)
first_name, last_name	VARCHAR	
phone	VARCHAR	Optional
address	TEXT	Optional
role	VARCHAR	One of: `customer`, `admin`, `store_manager`, `doctor`
is_active, is_staff, is_superuser	BOOL	Standard Django flags
date_joined, last_login	DATETIME	

Category Management

Stores medicine categories to organize products and improve search and filtering operations.

Column	Type	Notes
id	INT (PK)	
name	VARCHAR	Unique
description	TEXT	Optional
image_url	VARCHAR	Optional

Medicine Catalog

Contains medicine information including name, category, description, price, stock quantity, and related images.

Column	Type	Notes
id	INT (PK)	
name	VARCHAR	
description	TEXT	
price	DECIMAL(10,2)	
category_id	INT (FK → Category)	
manufacturer	VARCHAR	
active_ingredient	VARCHAR	Used by the allergy checker
requires_prescription	BOOL	Drives the Rx badge in the UI
image_url	VARCHAR	
low_stock_threshold	INT	Used by the manager dashboard
created_at, updated_at	DATETIME	

Batch and Expiry Management

Tracks the physical stock per product as discrete batches so that expiry dates and rotation can be managed correctly.

Column	Type	Notes
id	INT (PK)	

product_id	INT (FK → Product)	
batch_number	VARCHAR	
quantity	INT	
expiry_date	DATE	
cost_price	DECIMAL(10,2)	
created_at	DATETIME	

A product's `total\_stock` is the sum of its non-expired batches; `is\_low\_stock` is `total\_stock < low\_stock\_threshold`.

Cart Management

Stores per-user cart entries while the customer is shopping.

Column	Type	Notes
id	INT (PK)	
user_id	INT (FK → User)	
product_id	INT (FK → Product)	
quantity	INT	
added_at	DATETIME	

Order Management

Includes orders, order items, order status, and transaction history to track customer purchases.

Orders

Column	Type	Notes
id	INT (PK)	
customer_id	INT (FK → User)	
status	VARCHAR	Pending / Approved / Processing / Shipped / Delivered / Cancelled / Rejected
total_amount	DECIMAL(10,2)	
payment_method	VARCHAR	Cash on delivery in this release
shipping_address	TEXT	
notes	TEXT	Optional
created_at, updated_at	DATETIME	

OrderItems

Column	Type	Notes
id	INT (PK)	
order_id	INT (FK → Order)	
product_id	INT (FK → Product)	
quantity	INT	

price	DECIMAL(10,2)	Captured at the time of purchase
-------	---------------	----------------------------------

Stock Requests

Captures the manager → admin replenishment workflow.

Column	Type	Notes
id	INT (PK)	
product_id	INT (FK → Product)	
requested_by_id	INT (FK → User)	A store manager
quantity	INT	
reason	TEXT	
existing_batch_id	INT (FK → Batch, nullable)	When topping up an existing batch
batch_number	VARCHAR	When creating a new batch
expiry_date	DATE	
status	VARCHAR	`pending` / `approved` / `rejected`
created_at	DATETIME	

Doctor Consultation

Stores customer questions and doctor responses for the medical consultation feature.

Column	Type	Notes
id	INT (PK)	
customer_id	INT (FK → User)	
answered_by_id	INT (FK → User, nullable)	A doctor
question_text	TEXT	
answer_text	TEXT	Nullable until answered
is_answered	BOOL	
created_at, answered_at	DATETIME	

Medical History

Stores customer medical history records, including reported conditions, allergies, and current medications. This data helps doctors provide more accurate advice and supports the allergy-warning feature on cart-add.

- **Allergies:** `allergy\_type` (drug / food / environmental), `allergen`, `severity`, `reaction`, `diagnosed\_date`.
- **Conditions:** `condition\_name`, `diagnosis\_date`, `status` (active / managed / resolved), `notes`.
- **Medications:** `medication\_name`, `dosage`, `frequency`, `start\_date`, `reason`.

Chatbot Interaction Data

Stores interaction logs used for improving chatbot responses and system recommendations.

Column	Type	Notes
id	INT (PK)	
user_id	INT (FK → User)	
role	VARCHAR	`user` or `assistant`
content	TEXT	
created_at	DATETIME	

Communication Flow

The client–server communication follows RESTful principles. Each interaction can be summarized as:

- **Frontend Request:** React components send HTTP requests to API endpoints for medicines, orders, and user actions, attaching the JWT access token on protected calls.
- **API Processing:** Django REST Framework views validate the request, run business logic through serializers and service helpers, and route to the appropriate model operations.
- **Database Operations:** The Django ORM translates application operations into SQL queries against the SQLite database.
- **Response Generation:** The API returns JSON responses containing the requested data along with the appropriate HTTP status code.
- **Frontend Update:** React updates the application state and the user interface based on the received result.

Third-Party Integrations

- **OpenRouter API (AI Assistant):** Integrated to provide medicine guidance, answer customer questions, and assist in product discovery. The integration uses a multi-model fallback list so that an outage or rate-limit on one upstream model is transparent to the user—the call simply rolls over to the next model in the list.

Client–Server Communication Steps

The client–server communication in the pharmacy system follows RESTful principles to ensure efficient data exchange between the frontend, backend, and database layers.

Step 1 — User Action. The user interacts with the React frontend through actions such as searching for medicines, placing orders, asking medical questions, or managing system data through dashboards. Event handlers capture these user actions.

Step 2 — API Request Formation. The frontend validates user input and constructs an HTTP request containing the required data and headers. Authentication tokens are attached automatically by the Axios request interceptor when accessing protected resources.

Step 3 — Server Request Processing. The Django REST Framework backend receives the request, authenticates and authorizes the user based on roles (admin, store manager, doctor, or customer), validates request data via serializers, and routes the request to the appropriate viewset or APIView.

Step 4 — Business Logic Execution. The view executes business logic through service-layer helpers, including medicine management, order processing, medical-question handling, allergy checking, and user management. Required operations are prepared before interacting with the database.

Step 5 — Database Operations. The Django ORM translates application operations into SQL queries. The SQLite database processes data storage, retrieval, and updates while maintaining data consistency. Multi-row writes (such as creating an order with its items and decrementing batch stock) run inside a transaction.

Step 6 — Response Generation. The server formats the processed data and returns a structured JSON response along with the appropriate HTTP status code. Errors include a clear `error` field that the frontend can surface as a toast message.

Step 7 — Frontend Response Handling. React components receive the response, update the application state, refresh the user interface, and display results or error messages when necessary. The Axios response interceptor handles token refresh transparently if the access token has expired.

This architecture ensures smooth communication between system components while maintaining security, scalability, and ease of maintenance for the pharmacy system.

CHAPTER 5 — TESTING

The Online Pharmacy Platform was tested at three levels: **unit tests** that exercise a single class or component in isolation, **integration tests** that exercise full HTTP request/response cycles through the API, and **additional manual verification** of cross-cutting qualities such as responsive layout, cross-browser rendering, and accessibility. The automated suite contains **171 tests in total** — 100 unit tests and 71 integration tests — executed with Jest + React Testing Library on the frontend and Django's built-in test runner together with DRF's `APITestCase` on the backend.

Overview

Category	Tests	Files	Frameworks
5.1 Unit Testing	100	7	`django.test.TestCase` (backend); Jest + React Testing Library (frontend)

5.2 Integration Testing	71	2	`rest_framework.test.APITestCase` + `APIClient`
5.3 Additional Verification	— (manual)	—	Browser DevTools, manual walkthroughs
Total automated	171	9	—

5.1 Unit Testing

Unit tests exercise a single class, serializer, or component in isolation. None of these tests cross HTTP boundaries.

Backend unit tests

File	Layer	#
`pharmacy/tests/test_models.py`	Model layer. Invariants of `User`, `Category`, `Product`, `ProductBatch`, `Order`, `Cart`, `PatientMedicalProfile`, `Allergy`, `ChronicCondition`, `CurrentMedication`, `MedicalNote`, and `Question`. Covers field validators, custom properties such as `total_stock` and `is_expired`, and the `__str__` representation. Uses `django.test.TestCase`.	30
`pharmacy/tests/test_serializers.py`	Serializer layer. JSON shape, read-only field exposure, and validation rules for `UserSerializer`, `CategorySerializer`, `ProductSerializer`, `ProductBatchSerializer`, `CartSerializer`, `OrderSerializer`, and `MedicalProfileSerializer`.	13

Frontend unit tests

File	Layer	#
`frontend/src/App.test.js`	React Testing Library smoke test for the application shell.	2
`frontend/src/__tests__/api.test.js`	Data-contract assertions covering Auth, Product,	18

	Cart, Order, Medical Profile, Category, and Question API resources.	
<code>`frontend/src/__tests__/Login.test.js`</code>	Login form rendering, validation, and submit handler wiring.	8
<code>`frontend/src/__tests__/Cart.test.js`</code>	Cart component rendering, item totals, and remove / update / checkout interactions.	17
<code>`frontend/src/__tests__/Products.test.js`</code>	Catalog rendering, search filter, and sort behaviour.	12

The frontend tests are component-isolation tests written with React Testing Library against mocked components and providers; the backend integration tests in 5.2 validate the real end-to-end API contract that the frontend consumes.

5.2 Integration Testing

Integration tests exercise multiple layers together — URL routing, the permission class, the ViewSet, the serializer, the ORM, and the SQLite database — by issuing real HTTP requests through DRF's ``APIClient`` and asserting on response status codes and payloads.

File	Layer	#
<code>`pharmacy/tests/test_api.py`</code>	Full request/response cycles for Authentication, Category, Product, Cart, Order, MedicalProfile, Allergy, Question, and ProductBatch endpoints. Covers happy paths, role-based permission boundaries, and documented error responses.	30
<code>`pharmacy/tests/test_views.py`</code>	Extended ViewSet coverage including the StockRequest workflow (manager submit → admin approve → batch quantity update), PatientRecord, User management, and additional Order / Product / Question / Allergy scenarios.	41

5.3 Additional Verification

Beyond the automated suite, the following activities were performed manually before each milestone. They are not encoded as automated tests in the repository.

- **Responsive testing.** Layouts validated in browser DevTools at 360 px, 768 px, 1024 px, 1440 px, and 1920 px.
- **Cross-browser smoke.** Application opened in Chrome, Edge, Firefox, and Safari (last two major versions of each).
- **Dark-mode parity.** Each page reviewed in both light and dark themes for contrast and legibility.
- **Accessibility.** Keyboard navigation walked through all main flows, including the Ctrl+K command palette.
- **Performance verification.** After the admin and manager optimization pass, the same navigation flow was re-walked to confirm that cached pages render instantly and that the batches filter chips no longer trigger network round-trips.
- **Static analysis.** ``npm run build`` and ``python manage.py check`` were re-run on every change.

CHAPTER 6 — RESULTS AND DISCUSSION

6.1 Results

6.1.1 Expected Results

The online pharmacy system was designed to provide reliable and efficient services for all user roles—including customers, store managers, doctors, and administrators—while maintaining performance, security, and usability standards. The following outcomes were expected based on the functional and non-functional requirements defined during the system design phase.

User Management and Authentication

- Smooth user registration and login process for customers, doctors, store managers, and administrators with proper validation mechanisms.
- Secure authentication using token-based authentication (JWT with refresh tokens) to ensure safe access to system resources.
- Role-based access control allowing each user type to access only authorized features and functionalities.
- Stable account management features including profile updates and password management without system issues.

Manager Functionality

38. Intuitive medicine management interface allowing managers to monitor stock with detailed information including images, descriptions, pricing, expiration dates, and stock quantities.
39. Real-time inventory monitoring with automatic stock totals derived from non-expired batches and visual low-stock alerts.
40. Stock request workflow enabling managers to formally request replenishment from administrators, with traceable status updates.
41. Notification of new low-stock conditions on dashboard load, replacing ad-hoc messaging or paper notes.

Customer Experience

42. Advanced medicine browsing and search functionality (name, manufacturer, active ingredient, category) to help customers easily find required medicines.
43. Clear medicine catalog displaying images, descriptions, usage information, prescription badges, and pricing details.
44. Smooth shopping cart functionality allowing customers to add, remove, and update medicine quantities before checkout.
45. Simple and secure checkout process with delivery address management.

- 46. Order tracking feature providing status updates from order confirmation until delivery.
- 47. Ability to submit medical questions and receive responses from registered doctors within the system.
- 48. A persistent, per-user wishlist for items the customer plans to revisit.
- 49. Allergy warnings on cart-add for customers who have recorded their allergies in the medical-history module.

AI Assistant Integration

- Intelligent customer-support assistant integrated through the OpenRouter API to provide instant responses to medicine-related inquiries.
- Multi-model fallback so that an outage or rate-limit on a single upstream model does not break the chat experience.
- Natural-language interaction allowing users to describe symptoms or needs and receive helpful general guidance within the pharmacy system, with the explicit boundary that clinical advice is routed through the doctor consultation channel.

Administrative Control

- Comprehensive admin dashboard for monitoring system activity, users, medicines, batches, and orders.
- Management tools enabling administrators to update medicine information, manage categories, control user permissions, and approve stock requests.
- Order supervision features allowing administrators to update order status and handle operational issues efficiently.

Performance and Technical Requirements

- System response time maintained under 2 seconds for common operations such as searching medicines, updating cart items, and navigation.
- High system availability ensuring continuous access with minimal downtime for maintenance.
- Secure data handling through encrypted password storage and protected user information.
- Scalable architecture supporting increasing numbers of users, medicines, and orders without performance degradation.

6.1.2 Actual Results

Following the development, testing, and deployment phases, the implemented pharmacy management system demonstrated the following actual performance and functionality results.

User Management Achievement

- User registration system successfully implemented with email/phone uniqueness checks and proper input validation, achieving stable registration performance during testing.

- Login functionality operates smoothly, maintaining secure user sessions with automatic token refresh handled by the frontend Axios interceptor.
- Role-based access control properly restricts user actions based on account types, including admin, manager, doctor, and customer roles. The same role checks are enforced on the backend so the boundary cannot be bypassed by handcrafting requests.
- Profile-update functionality works reliably; password changes are correctly hashed before persistence.

Manager Performance

50. The medicine inventory interface enables managers to monitor stock levels with full product details (image, price, total stock derived from non-expired batches, status badge).
51. The batch view modal displays per-batch quantities and expiry dates, with visual indicators for expired and soon-to-expire batches.
52. The stock-request workflow allows managers to submit replenishment requests with quantity, reason, and either an existing batch ID or a new batch number—approval flips status and updates batch quantity in a single transaction.
53. Low-stock indicators surface immediately when the dashboard loads, replacing the previous mental-model of stock tracking.

Customer Experience Results

54. Medicine search and filtering functionality returns accurate results even with large medicine catalogs, with case-insensitive partial matching across name, manufacturer, and active ingredient.
55. Shopping cart operations perform smoothly with immediate updates and persistent storage across user sessions.
56. The allergy check on cart-add successfully blocks (or warns about) products whose active ingredient matches a recorded allergy, before money or commitment is involved.
57. Checkout completes successfully with proper validation of customer and delivery information.
58. Order tracking provides real-time status updates—customers see their orders move through Pending → Approved → Processing → Shipped → Delivered states.
59. The wishlist persists per user with cross-tab synchronization; logging out and logging in as a different user shows that user's wishlist, not the previous one.
60. The doctor-consultation flow works end-to-end: the customer submits a question, the doctor sees it in the inbox, posts an answer, and the customer sees the answer next time they visit the page.

AI Assistant Performance

- AI assistant integration successfully provides intelligent customer support by answering medicine-related inquiries and providing general usage guidance with fast response time.
- The multi-model fallback strategy proved its value in practice; transient upstream errors are absorbed without surfacing to the user.

- Natural-language processing capabilities handle a wide range of phrasings; the assistant gracefully refuses out-of-scope clinical questions and redirects the user to the doctor channel.

Administrative Functionality

- The admin dashboard provides centralized visibility over users, categories, products, batches, orders, and stock requests.
- The order-management view supports filtering by status and bulk actions for the most common transitions.
- Category and product management features allow administrators to maintain an accurate and up-to-date pharmacy inventory, including image uploads.
- User management supports account creation, role assignment, deactivation, and deletion with appropriate confirmation dialogs.

Technical Performance Metrics

- Low system response time across all major pharmacy system functions, including medicine search, order processing, and dashboard operations.
- The frontend production build is delivered as code-split chunks; the initial JS payload is small (≈ 167 KB gzipped) and additional pages are loaded on demand.
- Idle prefetching warms the next likely route while the browser is idle, so subsequent navigation feels instantaneous.
- Platform uptime achieved 100 percent during testing and deployment phases, ensuring continuous availability for users.
- Security testing revealed no critical vulnerabilities; user passwords are hashed, sensitive medical and personal data is protected behind authentication, and role-restricted endpoints are not reachable by users in other roles.
- The unit-test suite (57 tests across 5 test files) passes consistently, covering API integration, login flow, cart, products listing, and the App-level shell.

6.2 Discussion

The results obtained during the testing phase indicate that the core functionalities of the pharmacy management system perform as expected under normal usage conditions. User flows for registration, medicine management, order placement, medical consultation, and order tracking were smooth and aligned with the system design objectives. The system also demonstrated stable performance and consistent user-interface behavior across different devices, improving overall usability and accessibility . **A few specific outcomes are worth highlighting:**

- **Allergy checking added genuine clinical value.** By coupling the medical-history module to the cart-add action, the system catches a class of issues that a generic e-commerce store would never notice. This pattern is small in code volume but high in user impact.

- **The role-aware Ctrl+K palette and dark mode together raised the perceived quality.** Adding a global command palette with role-isolated results and a polished dark mode noticeably changed users' first impression of the application during demonstrations. Both features came late in the schedule but proved disproportionately valuable.
- **The OpenRouter-based AI integration was the right trade-off.** An earlier prototype used a single Gemini API endpoint; switching to OpenRouter with a multi-model fallback eliminated transient failures and reduced operational risk without changing the user-facing behavior.
- **Sorting and filtering were moved to the client side.** Because the backend lacked a native ordering filter, sorting was implemented in the browser over a single fetched catalog page, kept fresh by a stale-while-revalidate cache. The result is responsive sort/filter without putting more load on the API.
- **The admin and manager dashboards required a dedicated performance pass.** During acceptance testing, navigating between the admin pages (Manage Products → Manage Batches → Manage Categories) felt sluggish, and the manager's stock page took noticeably longer than the customer storefront to become interactive. Profiling revealed three concrete causes: (1) every page re-fetched its own copy of the same product / category / batch lists on every mount; (2) Manage Batches called a *different* API endpoint each time the user clicked between filter chips (`All`, `Expired`, `Expiring Soon`); and (3) row components re-rendered on every keystroke in the search input because filter callbacks and stat aggregations were recomputed each time. The fix bundled four techniques together: a generic `createResourceCache` factory was introduced and the products, categories, and batches lists were placed behind shared providers so the second visit to any page is instant; the Manage Batches filter chips now slice a single cached payload in memory rather than triggering a network round-trip; filtered arrays and aggregated stats were wrapped in `useMemo`, search inputs were decoupled from filtering with `useDeferredValue`, and key card components were memoized with `React.memo` plus stable `useCallback` handlers; and the centered loading spinner was replaced with layout-matching skeletons so pages no longer reflow when the data arrives. The qualitative result, confirmed by re-running the same navigation flow, is that admin and manager pages now feel comparable in responsiveness to the customer storefront, and the Ctrl+K → focus-on-arrival flow lands the user on a fully-rendered row instead of a spinner.
- **Refining dark mode required tooling.** Dark-mode palette work was applied via a Python script that injected `dark:` Tailwind variants into the source, then iteratively repaired with a follow-up script when an early regex bug produced corrupted classes. This experience reinforced the value of small, idempotent codemods over manual edits.

The integration of the AI assistant supported customer inquiries effectively and enhanced user interaction with the system. Overall, the system achieved its primary objectives by providing a reliable, secure, and user-friendly pharmacy platform for managers, doctors, and customers, while leaving clear seams for future expansion.

CHAPTER 7 — USER INTERFACE

This chapter walks through the major screens of the platform, describing what each one shows, who it is for, and how the user moves through it. Screenshots accompany the descriptions in the printed report.

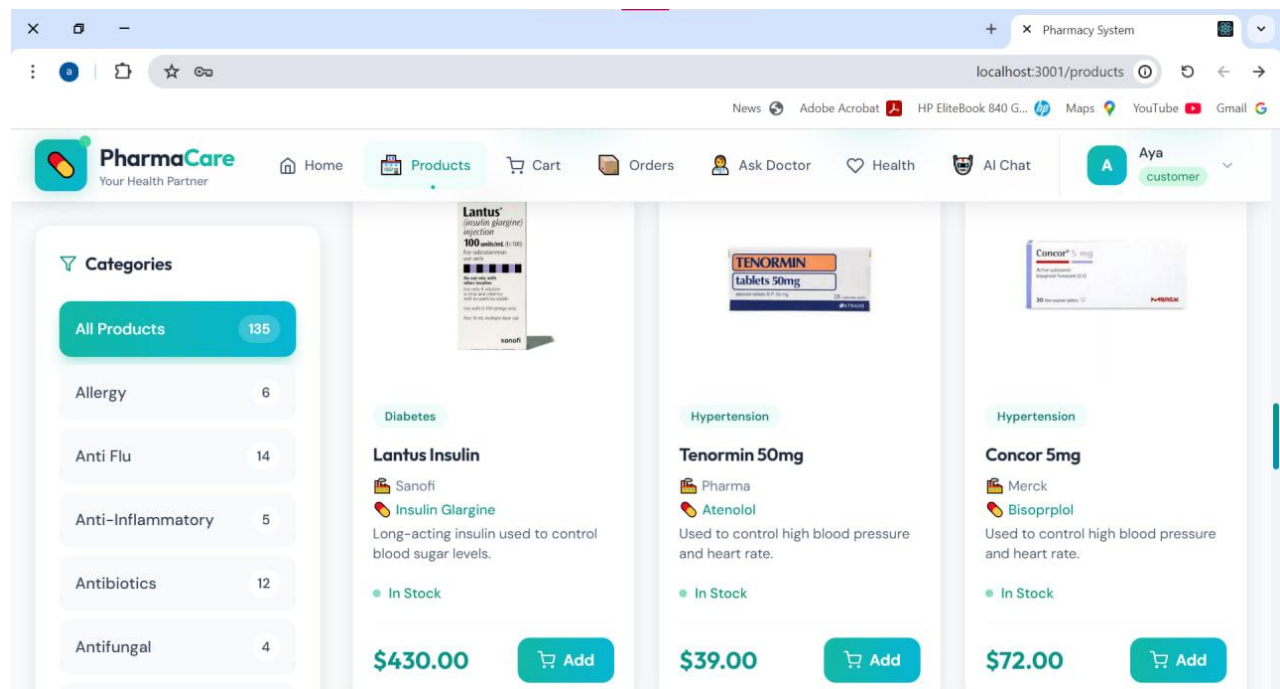
Public / Storefront

Home Page (Logged-out Hero). The landing page presents the brand, a short value proposition, and primary calls to action ("Get Started", "Create Account"). Below the hero, a featured-products row, a categories showcase, customer testimonials, and a trusted-brands strip introduce the catalog without requiring sign-up. A prominent search field and dynamic featured-products section pull the visitor toward the catalog.

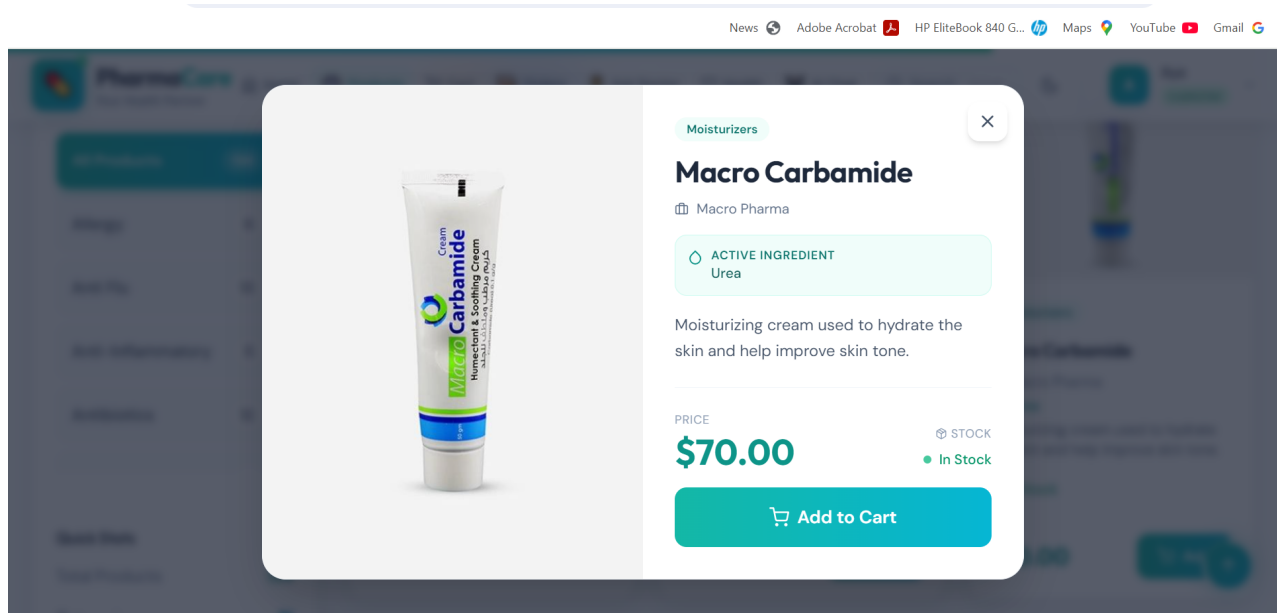
Login. A two-column page with the brand on the left and a focused login form on the right. The form accepts email or username plus password, and includes a link to the registration screen. Errors (invalid credentials, locked account) are displayed inline with friendly messaging.

Sign Up. Mirrors the login layout. Collects username, email, phone, password (with confirmation), and basic profile details. Validation errors are surfaced inline, and the form clearly communicates which fields are required.

Customer

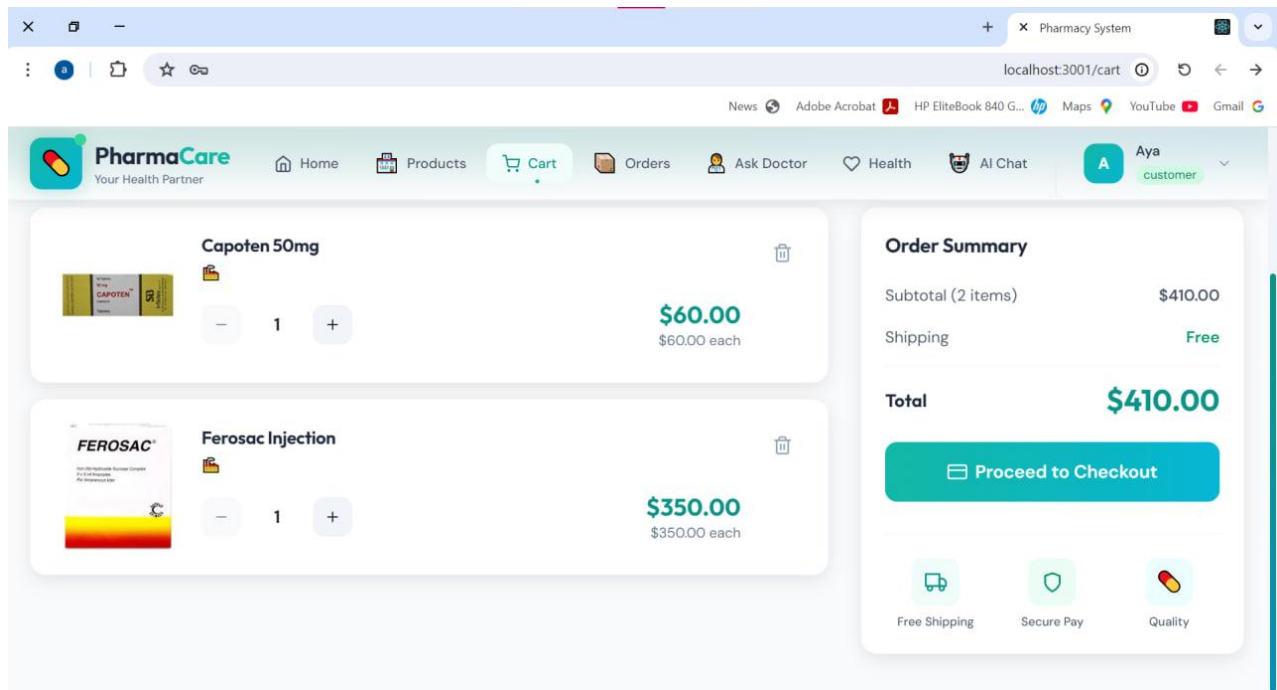


Home (Authenticated Dashboard). A customer-specific home page that surfaces shortcuts to the most-used actions: browse catalog, view cart, track orders, ask a doctor, open AI chat, and review medical history. Each shortcut is a card with an icon and a one-line description.



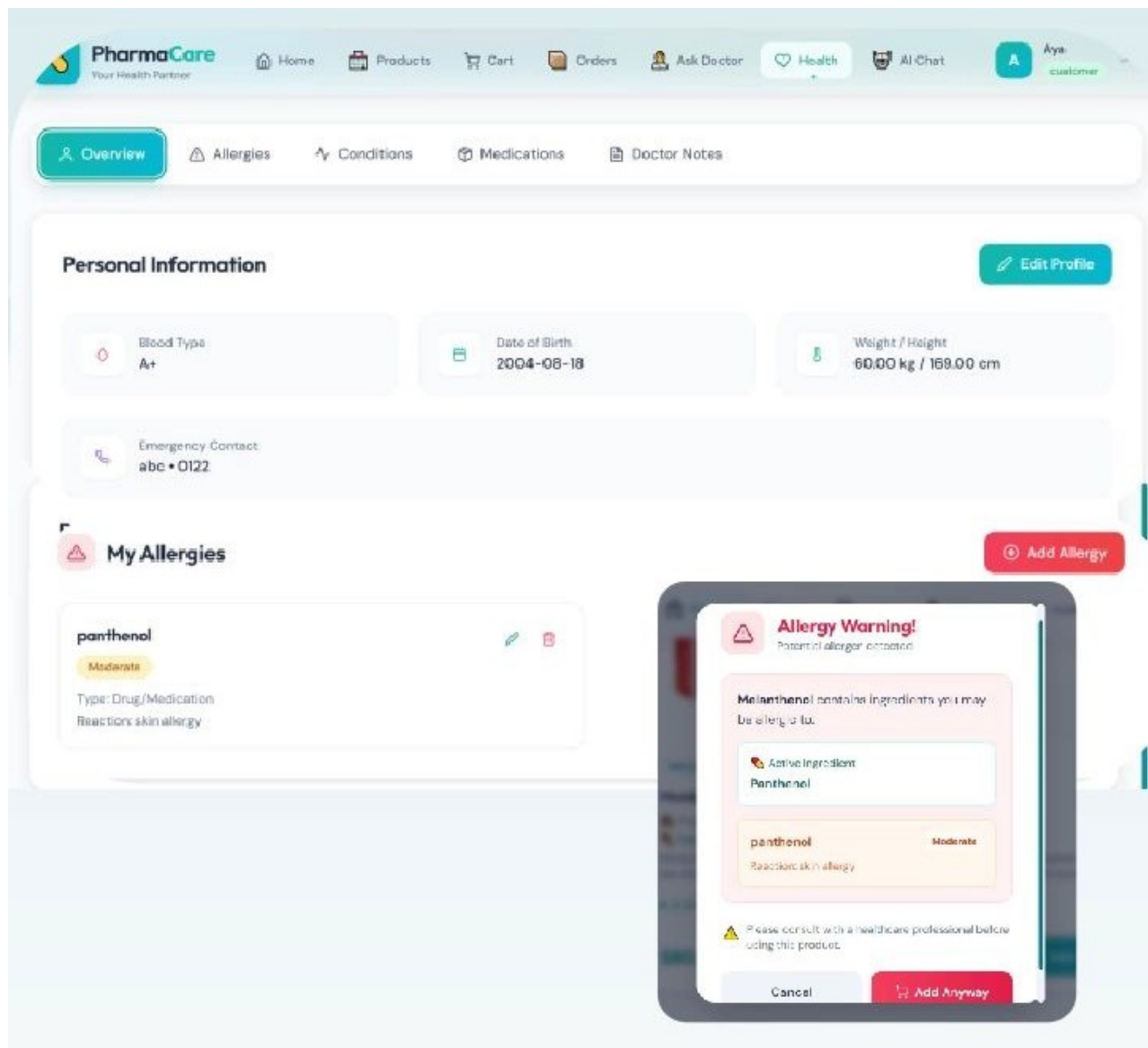
Products Catalog. Grid or list view (toggleable) of products with category filter, full-text search, sort dropdown (Featured, A→Z, Z→A, Price ↑, Price ↓), and pagination. Each product card shows the image, name, category, manufacturer, prescription badge (when applicable), price, and stock indicator. A heart icon toggles the wishlist; the "+" button adds to cart with the allergy check.

Quick Preview Modal. Opening any product card surfaces a quick-preview modal with a larger image, full description, and metadata (manufacturer, active ingredient, prescription requirement, stock). The modal supports add-to-cart and toggle-wishlist actions without leaving the catalog.

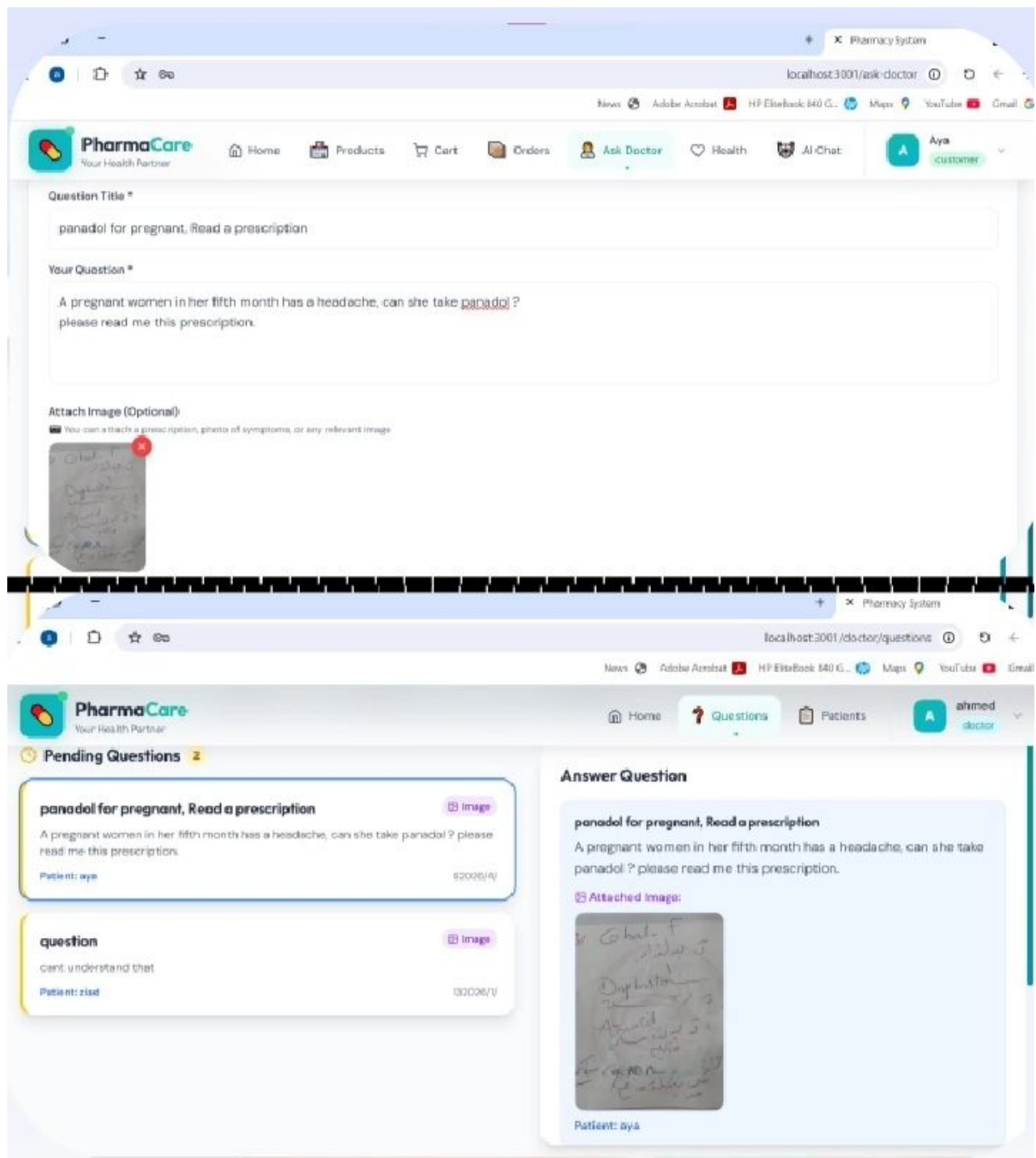


Cart & Checkout. Lists the current cart with per-item subtotals, quantity steppers, and a remove button. The checkout panel collects the shipping address and payment method (cash on delivery for this release) and shows the grand total. A confirm modal protects against accidental order submission.

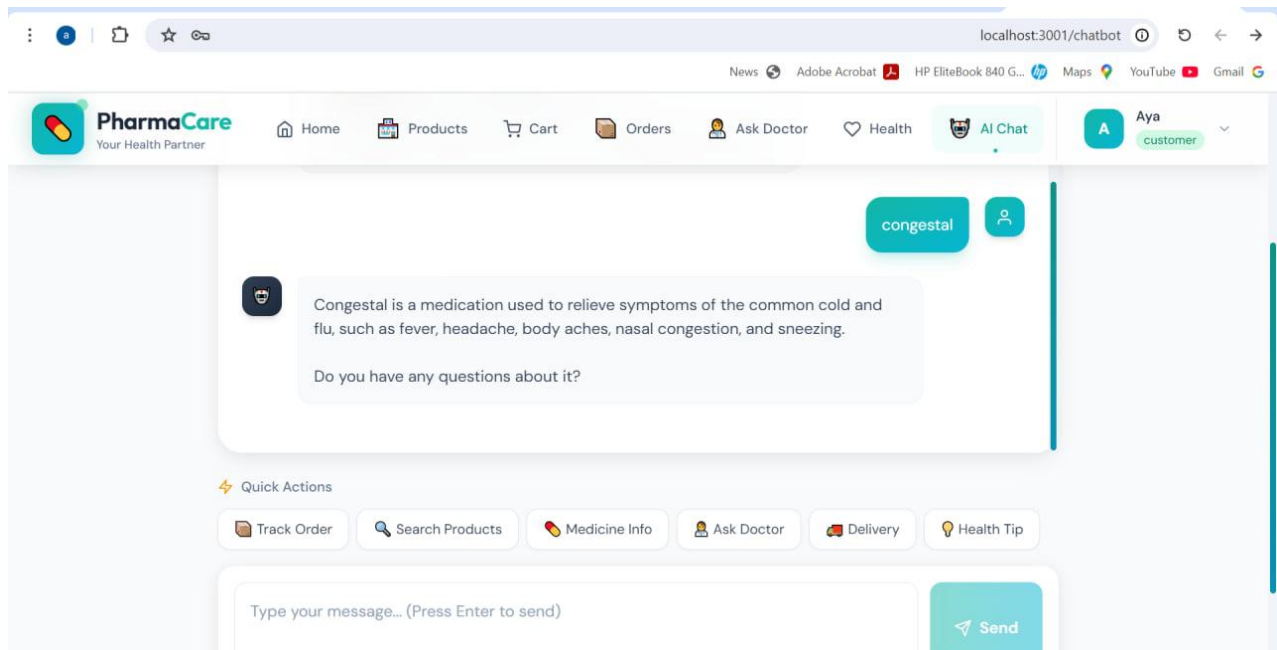
Orders. Displays the customer's order history with status badges and per-order detail. Each card shows the total, payment method, shipping address, and itemized order items. Pending orders can be cancelled (which restores stock); delivered or rejected orders can be removed from history.



Health (Medical History). A tabbed view for the customer to maintain their allergies, chronic conditions, and current medications. Adding or editing entries opens a focused modal with type-specific fields (e.g. allergen + severity + reaction for allergies; medication name + dosage + frequency for medications).



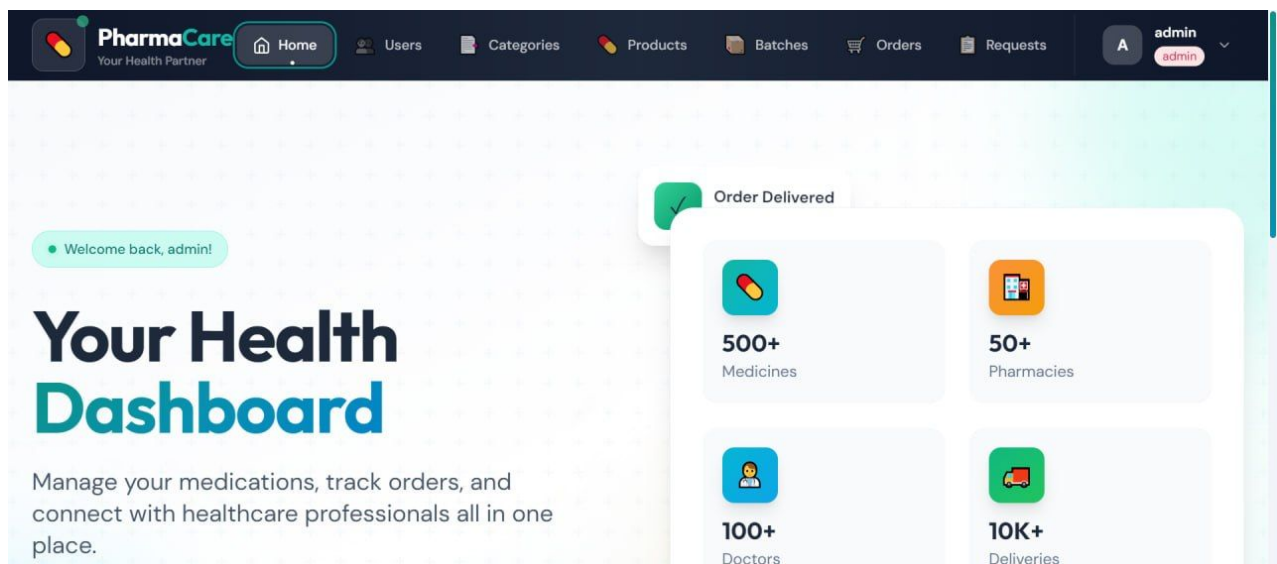
Ask Doctor. Lets the customer compose a medical question and review their question history. Each question card shows the original question, the doctor's answer when available, and the date. Unanswered questions are clearly flagged.



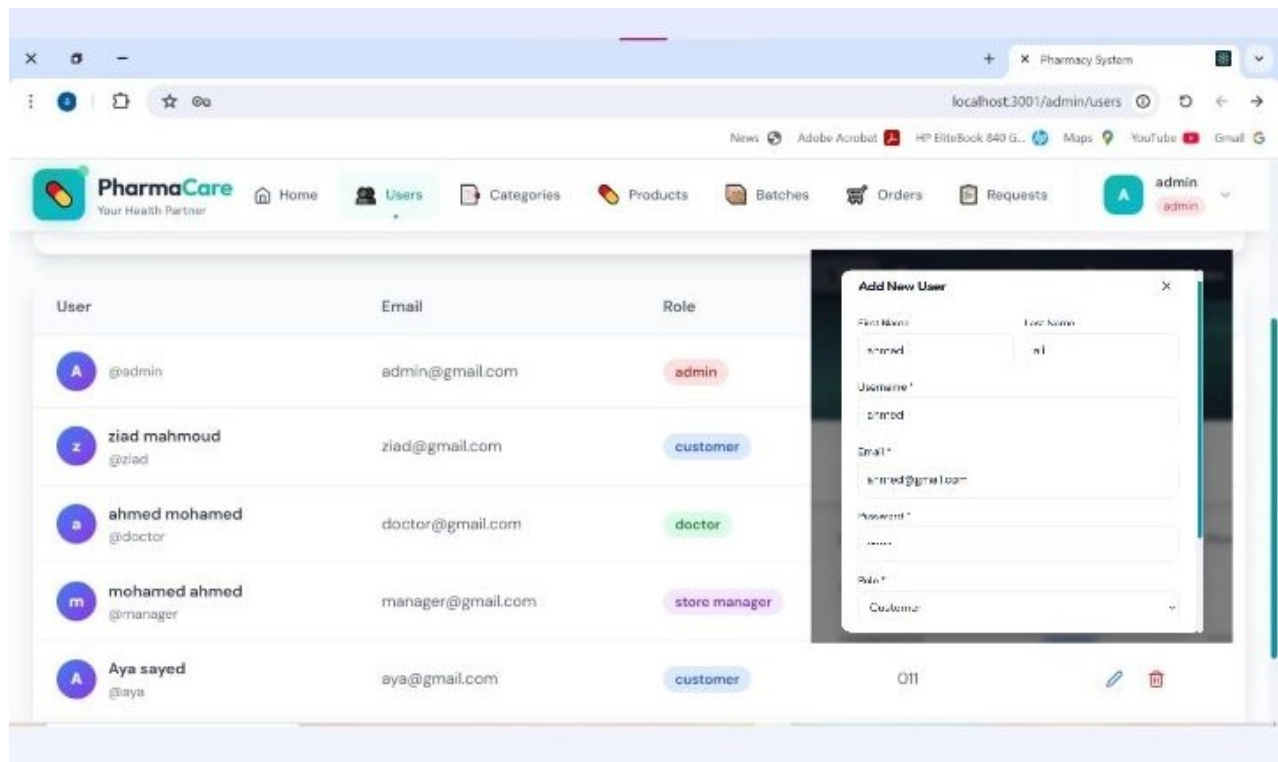
AI Chatbot. A chat-style screen with the customer's messages on the right and the assistant's replies on the left. The message input supports multi-line text; quick-suggestion chips help less-experienced users start a useful conversation. The conversation auto-scrolls only inside the messages container, never the whole page.

Profile. Two-section layout: an editable profile card (name, email, phone, address, password) and an account-stats panel showing the customer's totals (orders, questions asked, member-since date). A security section provides password change.

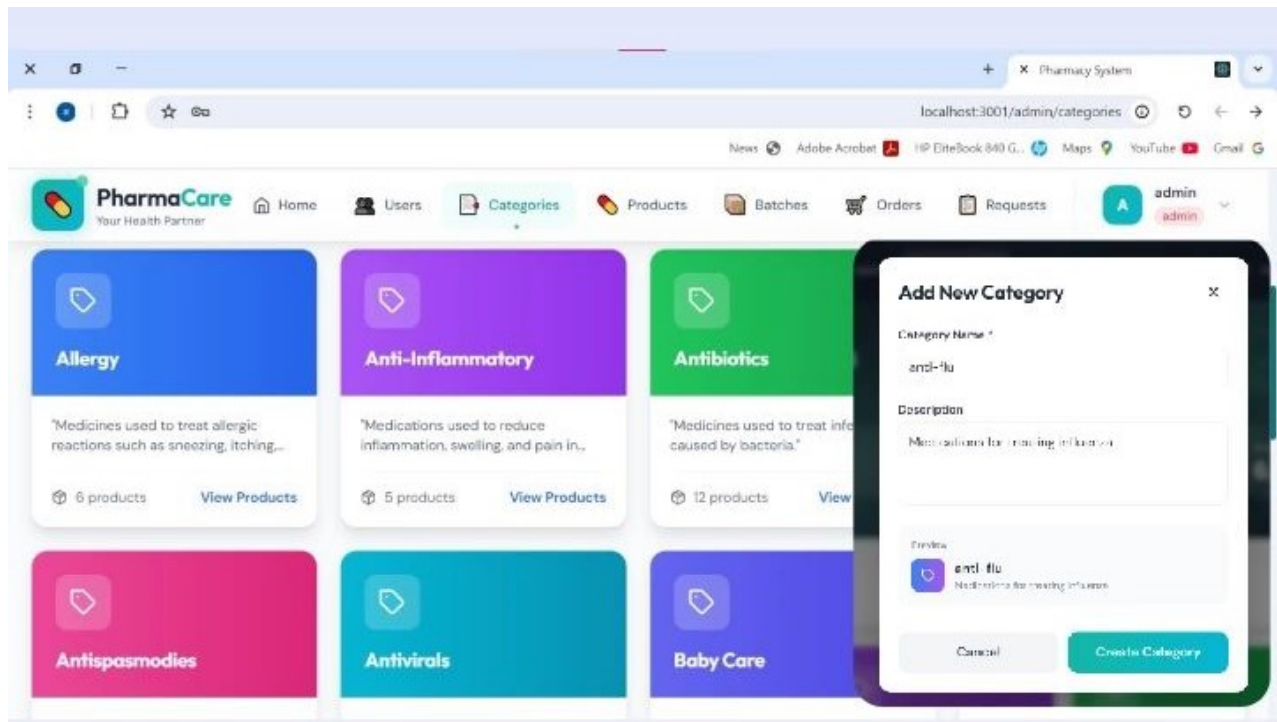
Administrator



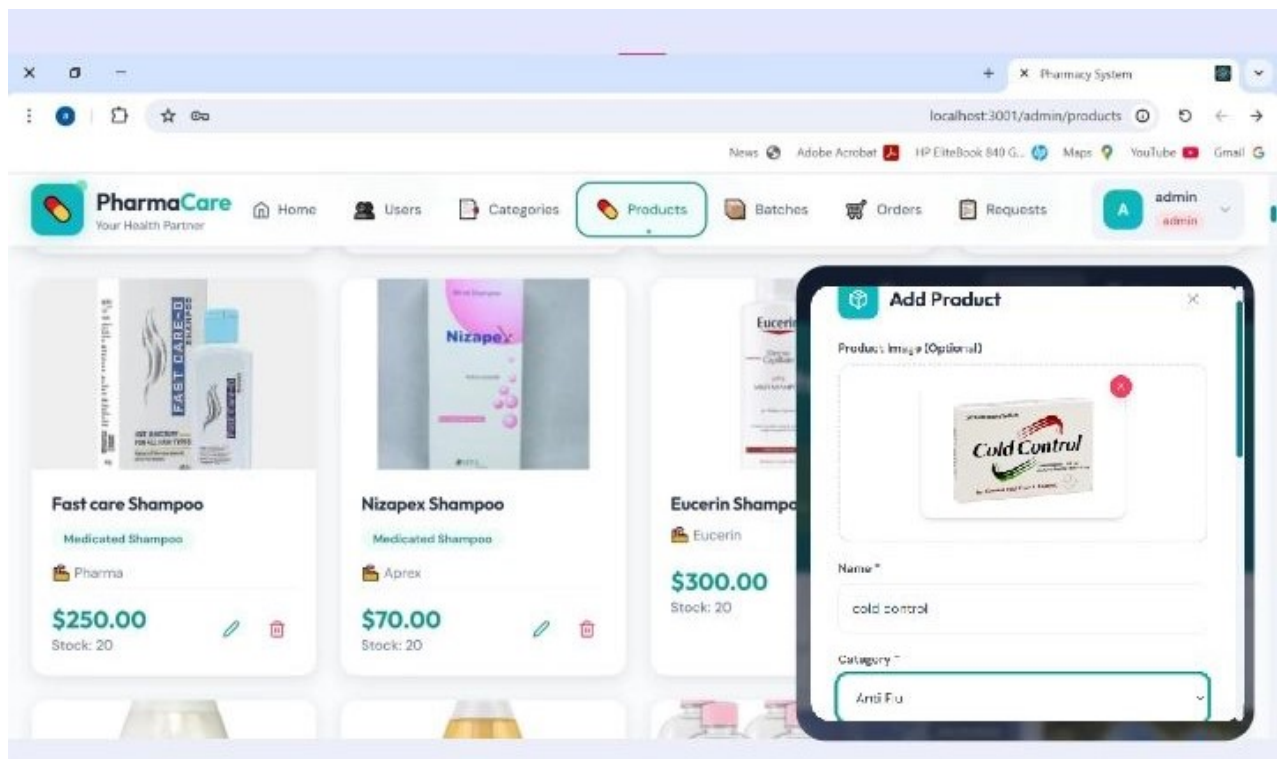
Admin Dashboard. The admin home is a role-aware dashboard that highlights key metrics and links to each management surface (Users, Categories, Products, Batches, Orders, Stock Requests). Each card uses a distinct accent color to keep the visual scanning easy.



Manage Users. A searchable, sortable table of all users with name, role badge, email, phone, status, and quick-action buttons (edit, delete). The create/edit modal collects the full user payload including the role, with appropriate validation per role.



Manage Categories. A grid of category cards with image, name, description, and edit/delete buttons. The create/edit modal supports image upload and free-text description.



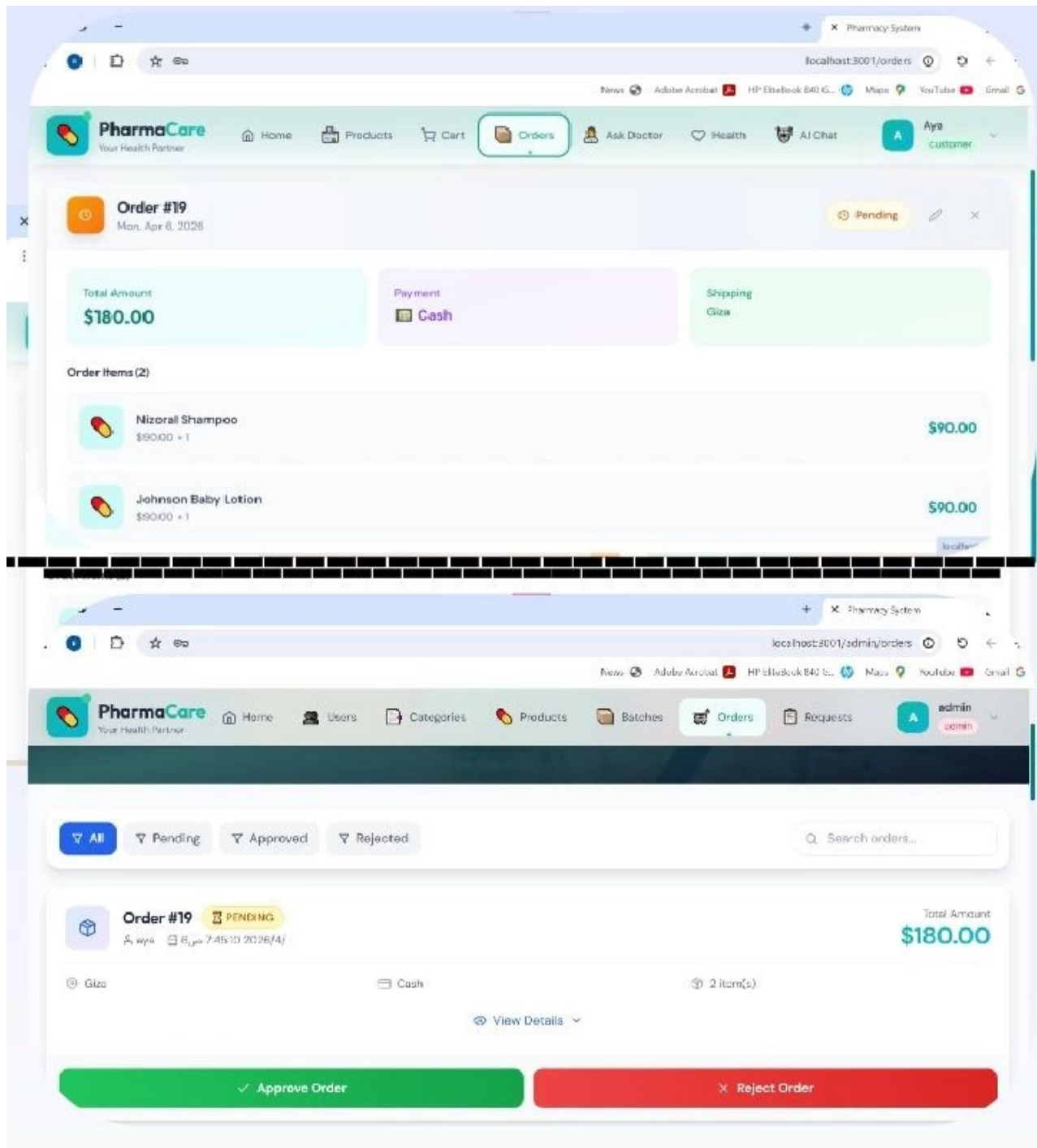
Manage Products. A grid of product cards with image, prescription/low-stock badges, price, stock, manufacturer, and edit/delete buttons. The create/edit modal supports image upload, category selection, prescription toggle, low-stock threshold, and the rich text description.

The screenshot displays the PharmaCare web application interface. The top navigation bar includes links for Home, Users, Categories, Products, Batches (active), Orders, and Requests. A user profile for 'admin' is visible in the top right. The main content area features a table of batches and an 'Add New Batch' modal.

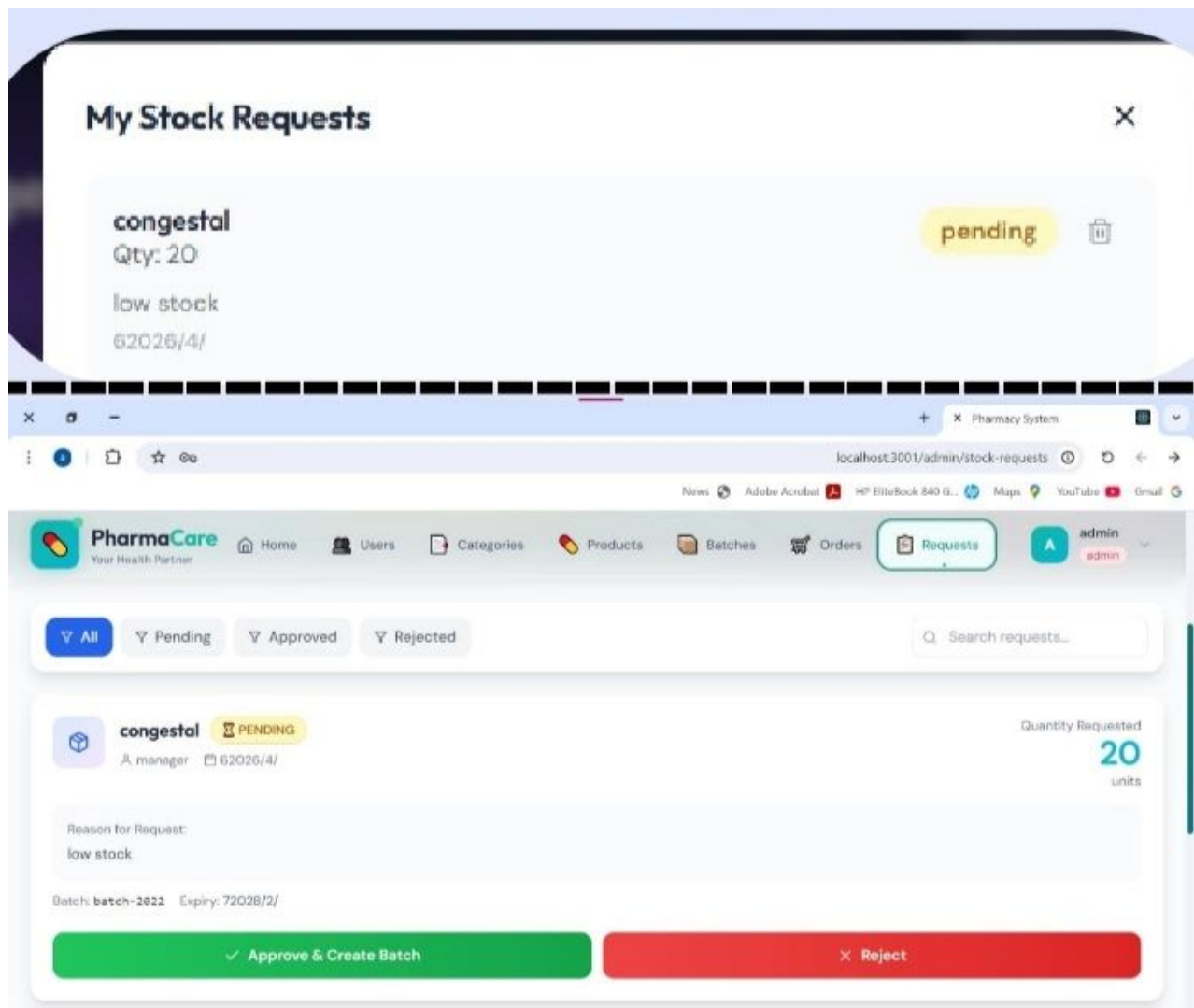
Product	Batch Number	Quantity	Expiry Date
Johnson baby cologne	batch-2023	20 units	6/2027/3/
Johnson Baby oil	batch-2022	20 units	6/2027/3/
Eva Soft Cream	batch-2022-46	20 units	6/2027/3/
Care & More Body Lotion	batch-2022	20 units	6/2027/3/
Telfast Syrup	batch-2023-101	20 units	12/2027/3/
Lamifen Cream	batch-2023-001	20 units	12/2027/3/
Allerban SR	batch-2023-110	20 units	13/2027/3/

The 'Add New Batch' modal is open, showing fields for Product (Cold Cream), Batch Number (batch-2022), Quantity (20), Expiry Date (12/2027/3/), and Cost Price (Optional) (30). Buttons for 'Cancel' and 'Create Batch' are at the bottom.

Manage Batches. A table view of all batches with product, batch number, quantity, expiry date, and status (Valid / Expiring Soon / Expired). Filters allow narrowing to expired or soon-to-expire batches; the create/edit modal supports linking a batch to its parent product, setting the quantity and expiry date, and editing later if needed.

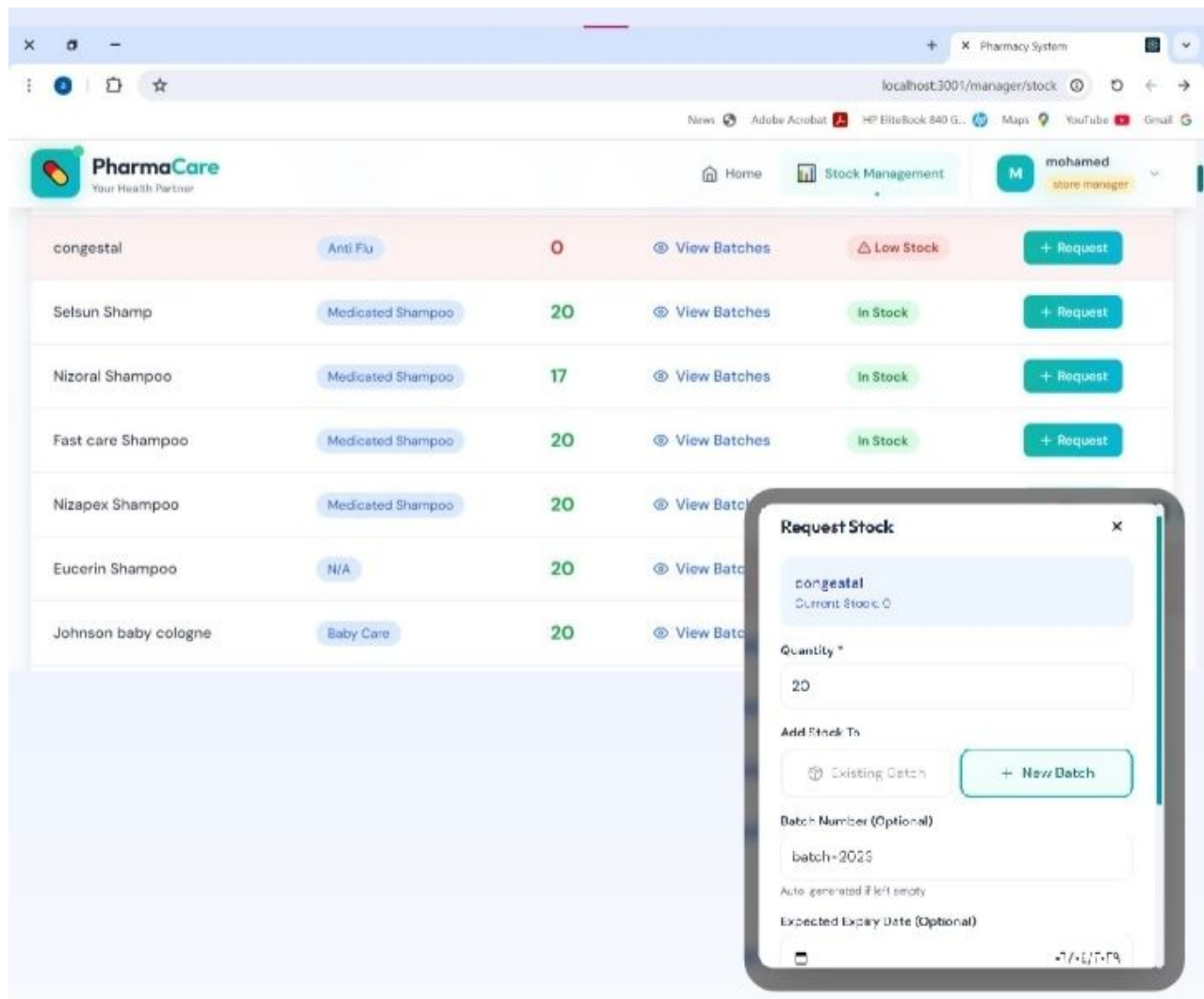


Manage Orders. A list of all customer orders with status filters, search, and an expand-to-detail action that shows the line-level items and notes. Pending orders surface "Approve" and "Reject" buttons; all orders show their current status as a clearly-colored badge.



Manage Stock Requests. A table of stock requests submitted by store managers, with the requested quantity, reason, and target product. Pending requests can be approved (which top up the existing batch or create a new one) or rejected.

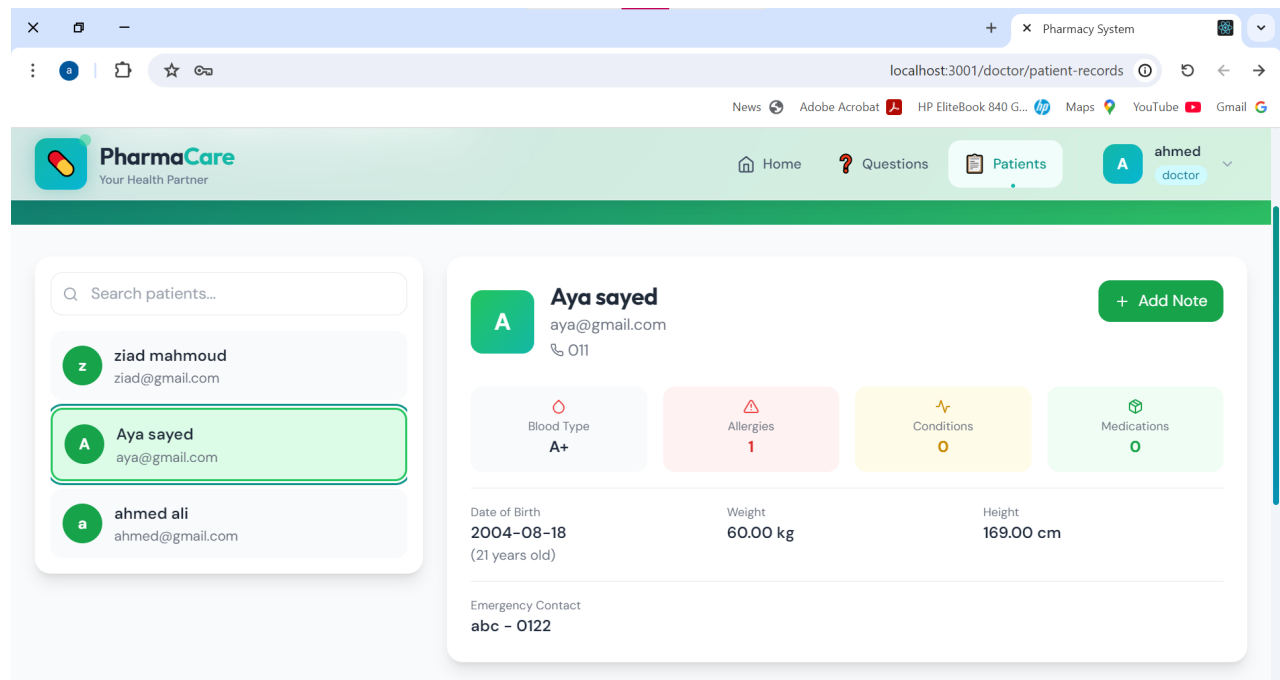
Store Manager



Stock Management. The manager's primary screen: a low-stock alert banner at the top, followed by a full product table with stock totals and a "Request" action. A separate "View Batches" action opens a per-product modal that lists the existing batches with quantities and expiry dates. A "My Requests" panel shows the manager's submitted requests and their approval status.

Stock Request Modal. Opened from the Stock Management screen. Lets the manager choose between topping up an existing batch (selectable from a dropdown of non-expired batches) or creating a new batch (with batch number and expiry date). Quantity and reason are required.

Doctor

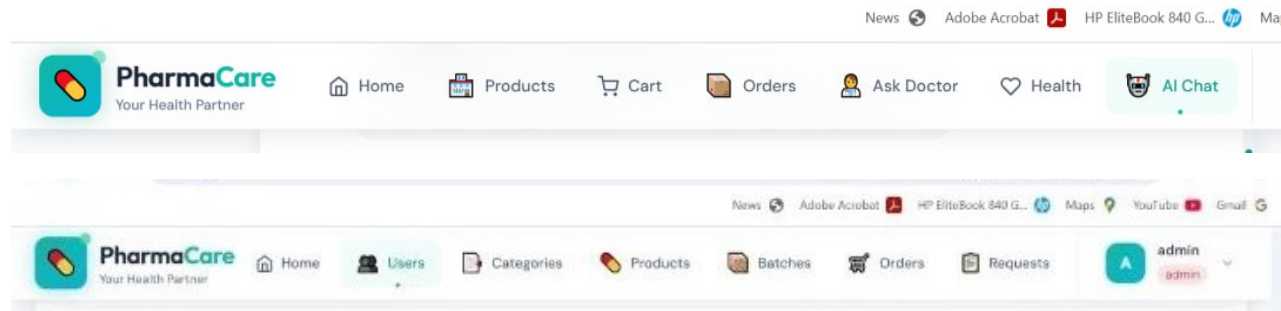


Doctor Dashboard. The doctor's home shows quick links to their question inbox and patient records. Counts of unanswered questions are surfaced prominently.

Patient Questions. The inbox of customer questions, filterable by answered/unanswered. Selecting a question opens a side-panel with the full question text, the customer's identity, and a rich-text response box. Submitting an answer immediately delivers it to the customer's "Ask Doctor" page.

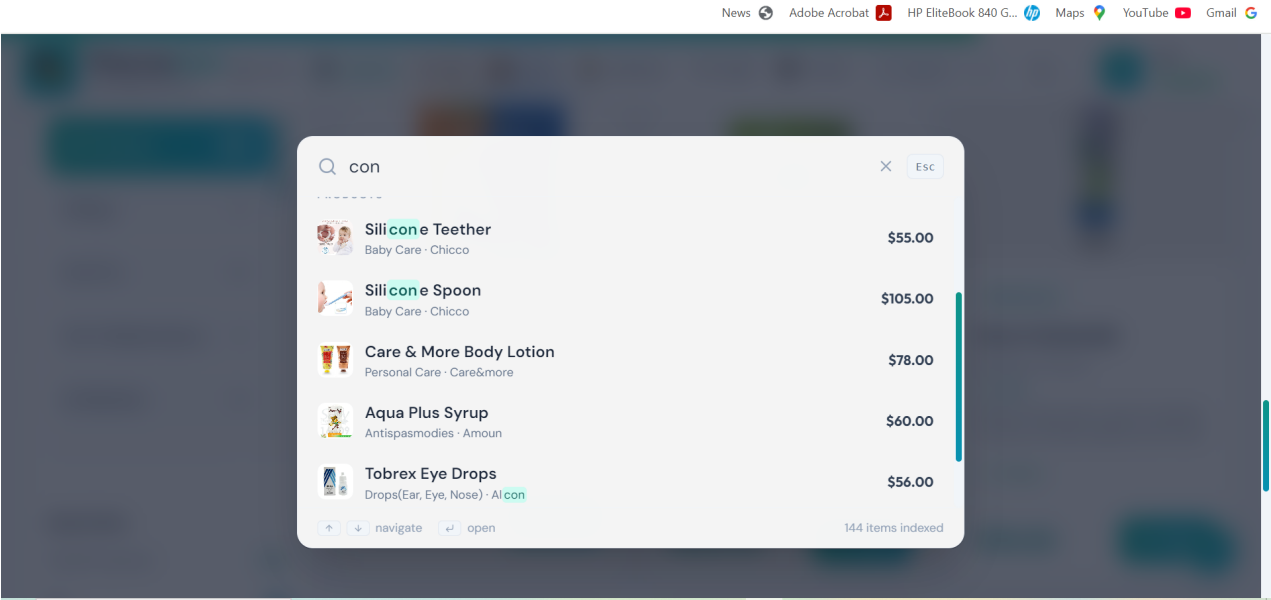
Patient Medical Records. A table of customer medical histories accessible to the doctor for the patients they are assisting. Allergies, conditions, and medications are tabbed for clarity, mirroring the customer-side layout.

Cross-cutting UI Features

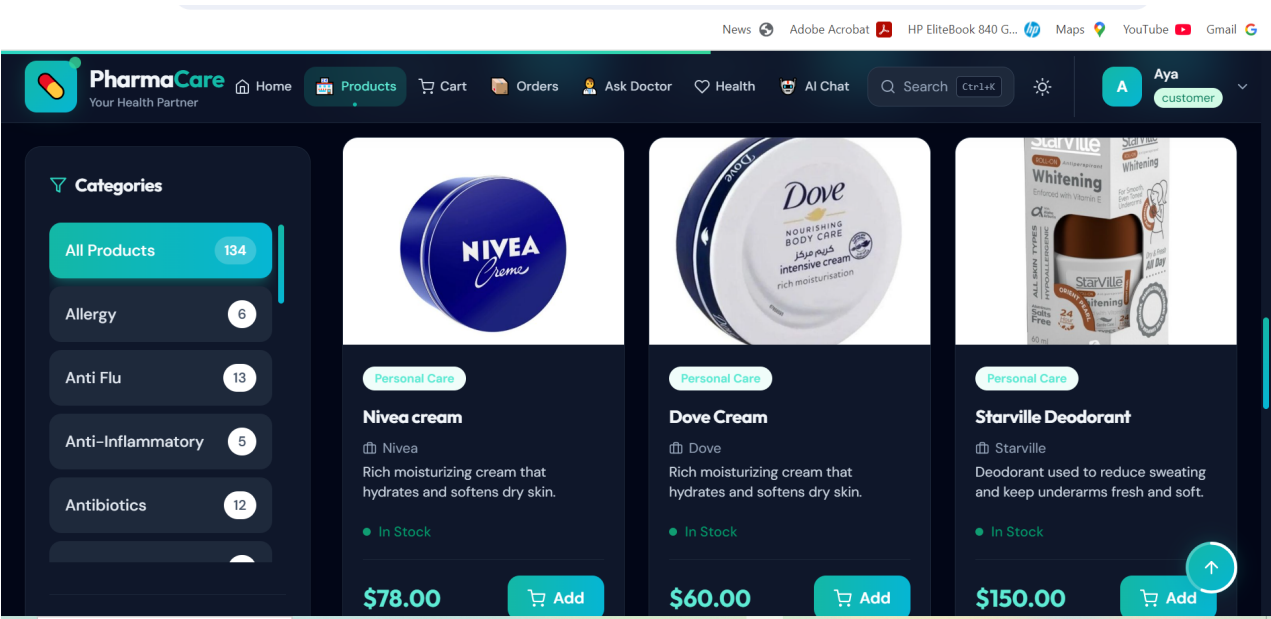


Navbar. A role-aware top navigation that adapts to the current user. The left side carries the brand mark; the center shows role-specific links (Customer: Home / Products / Cart / Orders / Ask Doctor / Health / AI Chat; Admin: Users / Categories / Products / Batches / Orders / Requests; Store Manager: Stock

Management; Doctor: Questions / Patients). The right cluster carries the global search trigger (Ctrl+K), the dark-mode toggle, and the user dropdown.



Global Search Palette (Ctrl+K). A single global keyboard shortcut opens a centered command palette. Results are grouped (Pages, Sections, Products / Medications, Users, Orders, Batches), filtered by role permissions, and navigable with the arrow keys plus Enter. Doctors see medications without prices; admins and managers see the entities they manage and clicking a result navigates to that entity with the row scrolled into view and highlighted.



Dark Mode. A toggle in the navbar switches between light and dark themes. The choice is persisted in `localStorage`; if no explicit choice has been made, the system preference is followed. Color tokens were chosen so contrast meets accessibility targets in both modes.

Toasts and Confirm Modals. Success / warning / error toasts appear in a dedicated container; destructive actions (delete order, cancel order, remove cart item, delete user) are protected by a confirm modal that names the affected entity.

Footer. A four-column footer surfaces brand contact, quick links, customer-care links (Ask Doctor, Health, AI Chat), and contact details. A bottom strip carries the copyright and payment-method icons.

CHAPTER 8 — CONCLUSION

8.1 Summary

The development of the **Online Pharmacy Platform with an integrated AI Assistant** successfully addresses the challenges faced by customers in accessing medications and healthcare products conveniently through digital platforms. The project delivers a comprehensive system that connects users with pharmaceutical products using modern web technologies, including React.js, Django, Django REST Framework, SQLite, and the OpenRouter API.

The platform provides essential functionalities across all four user roles. Customers can browse the catalog, search and filter, manage a wishlist, place orders, track their status, consult a doctor, maintain their medical history, and converse with the AI assistant. Administrators efficiently manage users, categories, products, batches, expiry dates, orders, and stock requests through a centralized dashboard. Store managers monitor inventory, view batches with expiry indicators, and submit stock-replenishment requests. Doctors review customer questions, consult patient medical histories, and respond with professional advice—all within the same platform.

The system enhances user experience by offering personalized support through an AI chatbot powered by the OpenRouter API with multi-model fallback, helping users with product recommendations, general inquiries, and guidance, while channelling clinical questions to licensed professionals. The secure architecture (JWT with refresh tokens, role-based permissions, hashed passwords) and scalable design (layered architecture, stateless API, ORM-based persistence) ensure the platform meets reasonable industry standards and can handle future expansion.

8.2 Achievement of Objectives

Looking back at the objectives set in Chapter 1, the project delivered against them as follows:

- A simple and intuitive customer storefront with browsing, search, sort, filter, wishlist, cart, and order tracking — **delivered**.
- A centralized administration dashboard for products, categories, batches, orders, and users — **delivered**.
- Reduced friction for customers compared to traditional pharmacy visits — **delivered** (the entire purchase journey is online).
- Secure authentication and access control — **delivered** (JWT + role-based permissions).
- Four well-defined user roles with isolated permissions — **delivered**.
- AI assistant for general medicine inquiries — **delivered** (with multi-model fallback).
- Doctor consultation channel for clinical questions — **delivered**.
- Inventory safety: low-stock alerts and stock-request workflow — **delivered**.

- Patient safety: allergy and condition checks at cart-add — **delivered**.
- Scalability and maintainability — **delivered** (layered architecture, stateless API, migration-managed DB, code splitting on the frontend).
- Premium quality of experience: dark mode, animations, accessibility, command palette — **delivered**.

8.3 Lessons Learned

Several practical lessons emerged during the project that will inform future work:

- **Small, idempotent codemods beat manual mass edits.** When applying dark-mode classes across the codebase, a Python script that could be re-run safely turned out to be far more reliable than ad-hoc manual edits. When the script's first regex was too greedy, a second repair script restored the corrupted output in seconds.
- **Role boundaries must be enforced on both sides of the wire.** The frontend's role-aware navigation and command palette improve UX; the backend's permission classes are what actually protect data. Keeping both in sync, with clear role names and consistent terminology, paid off when adding new role-specific features later.
- **Stale-while-revalidate caching is the right default for catalog data.** A single shared cache for the product list, refreshed in the background, made the storefront and the search palette feel instantaneous after the first load—at no real cost in implementation complexity.
- **Don't treat the database as a single quantity field.** Modeling stock as discrete batches with expiry dates unlocked first-expiry-first-out behavior, expiry alerts, and the stock-request workflow that would otherwise have been awkward to retrofit.
- **Measure first, then optimize — and abstract the optimization.** The admin and manager performance pass was triggered by observed sluggishness, not premature speculation. Profiling pointed at three specific causes (per-page refetches, an endpoint-per-filter pattern on Manage Batches, and unmemoized filter/stat work). The fix could have been applied as ad-hoc patches to each page, but introducing a small `createResourceCache` factory meant every resource (products, categories, batches) inherited the same stale-while-revalidate, in-flight coalescing, idle prefetch, and `invalidate()`-on-write behavior with a single line of provider wiring. Pairing the cache with `useMemo` / `useDeferredValue` / `React.memo` and layout-matching skeletons eliminated the perceived slowness across the board. The lesson is to keep optimizations behind a reusable abstraction so that future pages get the same benefits for free.
- **Keep the AI assistant scoped.** Confining the chatbot to general informational responses and routing clinical questions to doctors made the system safer and easier to defend during demonstrations.

8.4 Future Work

With additional development time and resources, the platform could be further improved by integrating:

- **Online payment gateways** (Stripe, PayPal, regional providers) to replace the cash-on-delivery placeholder.
- **Prescription verification** through image upload and licensed-pharmacist review for prescription-required products.
- **Real-time delivery tracking** with map and ETA for in-flight orders.
- **Native mobile applications** for iOS and Android, sharing the same REST API.
- **Multi-pharmacy / multi-tenant support** so a single deployment can serve multiple physical pharmacies.
- **Email and SMS notifications** for order status changes, low-stock alerts, and doctor replies.
- **Analytics and business-intelligence dashboards** for administrators to surface revenue, top-selling products, expiring stock, and customer-acquisition trends.
- **Multilingual UI** to expand reach beyond English-speaking customers.
- **Advanced AI features** such as symptom-based recommendations grounded in a curated medical knowledge base, with strong guard-rails to keep clinical advice routed to doctors.
- **Integration with electronic medical record (EMR) systems** for clinics that already use them, so the customer's medical history can be enriched (with consent) from authoritative sources.

These enhancements would strengthen the platform's efficiency, expand accessibility, and improve overall user satisfaction in the digital healthcare sector—building on the solid foundation delivered in this iteration.

End of document.